

Large Scale Data Retrieval in e-Health Applications

Masterarbeit

zur Erlangung des akademischen Grades

Master of Science (M.Sc.)

Eingereicht bei:

**Fachhochschule Kufstein Tirol Bildungs GmbH
Web Communication and Information Systems**

Verfasser/in:

Martin Schipflinger

1310738955

Erstgutachter : Prof. (FH) Dr. habil. Mario Döller

Zweitgutachter : Prof. (FH) Dr. Michael Bredel

Abgabedatum:

02. November 2015

EIDESSTATTLICHE ERKLÄRUNG

Ich erkläre hiermit, dass ich die vorliegende Masterarbeit selbstständig und ohne fremde Hilfsmittel verfasst und in der Bearbeitung und Abfassung keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe. Die vorliegende Masterarbeit wurde nicht anderweitig für Prüfungszwecke vorgelegt.

Kufstein, den 02. November 2015

Martin Schipflinger

ZUSAMMENFASSUNG

Diese Arbeit beschäftigt sich mit dem Thema der Datenabfrage auf große Datenmengen in e-Health Anwendungen. Die International Agency for Research on Cancer der World Health Organization betreibt derzeit eine Anwendung zur Sammlung von Ernährungsdaten um durch Auswertungen, Rückschlüsse über die Auswirkung von Ernährung auf die Bildung von Krebs zu erhalten. Dieses System ist derzeit sehr langsam und es lässt eine zumutbare Verwendung durch lange Datenladezeiten nicht zu. Ziel der Arbeit ist eine Überprüfung dieser Software auf Ihre Umsetzung beziehungsweise hinsichtlich möglicher Optimierungen. Dazu wird zuerst im theoretischen Teil aktuelle Literatur vorgestellt und diskutiert um einen Überblick über die derzeit Verwendeten best practices in Punkto Datenabfrage, Indizierung, Datenbank Typen und möglichen Hardwareengstellen zu erhalten. Anschließend wird in einer Case Study das bereits erwähnte System vorgestellt und bezüglich ihres Optimierungspotentials untersucht. Abschließend werden Anregungen und Möglichkeiten zur Verbesserung angeführt.

ABSTRACT

This work deals with large scale data retrieval in e-health applications. The International Agency for Research on Cancer of the World Health Organization is currently maintaining a system for collecting global nutrition facts to analyze the data and therefore make conclusion on how nutrition is affecting the development of cancer. This system is currently very slow and a usage is due to high waiting times for data retrieval not reasonable. The aim of this thesis is a review of this system regarding implementation and optimization. Therefore the theoretical part discusses best practices for data retrieval, indexing, database types and hardware bottlenecks. This best practices are then compared in a case study which firstly introduces the previous mentioned application and gets afterwards examined regarding potential for optimization. Finally suggestions and ideas are presented to improve the system in order to get a maintainable and performing application.

CONTENTS

i	INTRODUCTION	1
1	INTRODUCTION	2
1.1	Motivation	2
1.2	Goal	2
1.3	Methodology	3
1.4	Structure	3
ii	RELATED WORK AND THEORY DISCUSSION	4
2	PREFACE	5
3	LIMITING HARDWARE FACTORS	6
3.1	Memory	7
3.2	Processor	9
3.3	Network	9
3.4	Hard Disk	11
4	DATABASE TYPES	14
4.1	Relational Databases	14
4.2	NoSQL Databases	16
5	SQL QUERIES	19
5.1	Internal Query Processing	19
5.2	Query Performance	21
5.3	Optimization	26
5.3.1	SQL Query Best Practices	26
5.3.2	SQL Query Optimization	28
6	INDEXES	34
6.1	Index Types	34
6.1.1	Heap Tables	34
6.1.2	Clustered Index	35
6.1.3	Non-Clustered Index	35
6.2	Index Strategies	36
6.2.1	Heap Tables	37
6.2.2	Clustered Index	37

6.2.3	Non-Clustered Index	39
6.3	Index Maintenance	43
7	SUMMARY	47
iii	USE CASE STUDY: APPLICATION AND IMPROVE- MENT	49
8	E-SMP APPLICATION	50
8.1	Context and Objectives	50
8.2	Architecture	51
8.3	Queries	55
8.4	Issues	57
9	E-SMP FINDINGS AND IMPROVEMENTS	59
9.1	Test Environments	59
9.2	Stored Procedure IMGetInterviews	62
9.2.1	High Runtime Problem	62
9.2.2	Possible Improvements	63
9.3	SQL Server Version	69
9.3.1	Outdated SQL Server Version	69
9.3.2	Possible Solutions	69
9.4	Interview Pagination	72
9.4.1	Procedure Call at Each Page	72
9.4.2	Possible Solution	74
9.5	Webarchitecture	74
9.5.1	Plug-in technology security issues	74
9.5.2	Possible Solutions	76
10	SUMMARY	78
iv	CONCLUSIONS AND FUTURE WORK	80
11	CONCLUSIONS	81
12	FUTURE WORK	83
	BIBLIOGRAPHY	84

LIST OF FIGURES

Figure 1	SQL Server Activity Monitor	6
Figure 2	Windows Network Monitor	10
Figure 3	Benchmark Hard Disk Drive vs. SSD Drive	13
Figure 4	Relational Database Table	15
Figure 5	SQL Server Query Execution Optimiza- tion [7]	20
Figure 6	SQL Server Extended Event Session . . .	23
Figure 7	Query Response Time [37]	24
Figure 8	SQL Server Execution Plan	25
Figure 9	Invalid Search Argument vs Valid Search Argument	31
Figure 10	B+ Tree Structure [5]	36
Figure 11	Non-Clustered Index Seek	41
Figure 12	Multiple Columns Index Pattern	42
Figure 13	e-SMP Application Front-End View	51
Figure 14	e-SMP Highlevel Architecture	52
Figure 15	e-SMP Code Architecture	53
Figure 16	e-SMP Database Architecture	54
Figure 17	LINQ Architecture [33]	56
Figure 18	Note Retrieval Time	58
Figure 19	Virtual Box Test Environment	60
Figure 20	Request Time Table for Note Retrieval . .	62
Figure 21	Retrieve Notes	63
Figure 22	IMGetInterviews Insert Into itv_note . . .	68
Figure 23	IMGetInterviews Improved Execution Plan	69
Figure 24	SQL Server 2014 Improved Timing	70
Figure 25	Interview Pagination	74
Figure 26	Runtime Comparison of Stored Procedure IMGetInterviews	78

LIST OF TABLES

Table 1	Memory Performance Counters	8
Table 2	Processor Performance Counters	9
Table 3	Network Performance Counters	10
Table 4	Disk I/Os by RAID Levels	12
Table 5	Interview Notes Retrieval Time	66
Table 6	Activity Monitor Outcome of IMGetInter- views	67
Table 7	Interview Notes Retrieval Time SQL Server 2014	71
Table 8	Vulnerability Database Entries for the Sil- verlight plug-in	75
Table 9	CVSS Score Severity Level	76

LIST OF ABBREVIATIONS

AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
CLI	Command Line Interface
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DBMS	Database Management System
DLL	Dynamic Link Library
e-SMP	e-Standardized Methodological Platform
GB	Gigabytes
GHz	Gigahertz
GUID	Globally Unique Identifier
GUI	Graphical User Interface
HTML	Hyper Text Markup Language
IARC	International Agency for Research on Cancer
IM	Interview Manager
I/O	Input/Output
IT	Information Technology
JS	JavaScript
LINQ	Language Integrated Query

MS	Maintenance and Settings
MVC	Model-View-Controller
NIST	National Institute of Standards and Technology
NoSql	Not only SQL
NPAPI	Netscape-Plug-in API
OS	Operating System
RAID	Redundant Array of Independent Disks
RPC	Remote Procedure Call
SAN	Storage Area Network
SARG	Search Argument
SC	Security and Communication
SOAP	Simple Object Access Protocol
SQL	Structured Query Language
SSD	Solid State Drive
T-SQL	Transact-SQL
varchar	variable character length
XAML	Extensible Application Markup Language
XML	Extensible Markup Language

LISTINGS

Listing 1	SQL Query Example	24
Listing 2	SQL Query without Identifiers	29
Listing 3	SQL Query with Identifiers	29
Listing 4	Late Row Reduction	29
Listing 5	Early Row Reduction	30
Listing 6	Union Select	32
Listing 7	Case When	32
Listing 8	SQL Query with statistics on	41
Listing 9	CompiledQuery Example	56
Listing 10	Procedure Call Example	60
Listing 11	Insert Into itv_Note from IMGetInterviews	66
Listing 12	IMGetInterviews Table Variable Improve- ment	68
Listing 13	Varied Request Parameters for IMGetInter- views	72
Listing 14	Request Parameters for IMGetInterviews	73

Part I

INTRODUCTION

INTRODUCTION

As the amount of data, stored in any kind of database, increases steadily, some suitable mechanisms for retrieving large amounts of data have to be used. In collaboration with the International Agency for Research on Cancer ([IARC](#)) I try to show in this theses what practices can be taken to maintain a huge data storage for storing dietary data for a Global Dietary Surveillance project. During this work I want to show how data retrieval can be managed, to efficient retrieve data in an application and I want to answer the question: Does the in the e-Standardized Methodological Platform ([e-SMP](#)) application used data retrieval technologies follow best practices and are they suitable for the desired purpose.

1.1 MOTIVATION

As a software engineer and developer, I often have to deal with data stored in databases. Sometimes the amount of data stored, reaches a limit where timely processing gets difficult. Therefore in this thesis I try to figure out, what can be done in terms of data retention and data retrieval to maintain a performing data storage.

1.2 GOAL

The goal of this theses is to show how data maintenance should be done and showing up some possible solutions on improving the from the [IARC](#) maintained [e-SMP](#) application in order to get a usable and maintainable data analysis system.

1.3 METHODOLOGY

To reach the above stated goal, I firstly review the current state-of-the-art best practices for storing and retrieving data. I further show which databases are suitable for the desired purpose and point out how query design and index usage can affect the effectiveness of data retrieval. Next I execute some experiments based on the from the [IARC](#) provided platform, to show up some possible improvements to the application.

1.4 STRUCTURE

This work is separated into three major parts. The first part is Related Work. Here current literature and best practices are discussed to get a overview over what can be done to improve performance for data retrieval. The second part is a use case study based on the [e-SMP](#) application where the application and the current performance issues get explained and improvements based on experiments are shown. The last part are conclusions and future work. Here the in the use case developed improvements get critical reviewed and some advise on future work is given.

Part II

RELATED WORK AND THEORY DISCUSSION

PREFACE

This part of the work is showing the most important impacts and the most common errors when having to deal with data retrieval performance. Therefore this section is divided into four major content chapters. The first chapter is dedicated to potential hardware bottlenecks. It shows what can be done in order to identify and prevent hardware bottlenecks. The second chapter deals with database types. It shows which database types are available and which can be used for the in the case study introduced application. It supports the decision making for choosing the right database for the right purpose. The third chapter concerns itself with SQL query insights. It shows how queries should be issued in order the improve query reuse. The last content chapter is about indexes. As poor indexing is one of the most common issues when having to deal with a weak performance, advises on how and which indexes should be taken are given.

There are a couple of other things which can affect database performance, like for example the data model. But as these are the most common issues, which have the highest potential to increase data retrieval performance, this work focuses on the above mentioned four major impacts.

LIMITING HARDWARE FACTORS

Hardware is building the basic of a computer system. Therefore the underlying hardware is very important to the performance of a system. In this section we will have a look on the four most important factors concerning performance of an SQL Server namely:

- Memory
- Processor
- Network
- Harddisk

If a system is facing a poor performance in terms of data retrieval, you have to make sure, before analyzing the internals of an application like indexes, queries and execution plans, that the reason for your performance problems is not a bad or inadequate hardware. Thus this section will present important information to analyze a system in terms of hardware so that hardware bottlenecks can be found and worked out. In SQL Server you can check the performance over the Activity Monitor (Figure 1). This monitor provides different counters that provide you with information about the current state of your SQL Server instance.

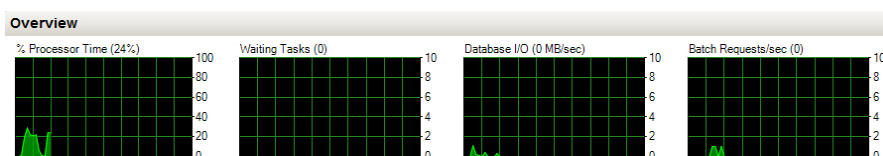


Figure 1: SQL Server Activity Monitor

3.1 MEMORY

Memory is a important hardware resource as every data, requested by an agent, gets loaded into the memory. Further many other operations such as sorting or joining tables use the fastness of the main memory. Therefore it is important to have enough memory available to balance the current work load. If not so a memory bottleneck is emerging and this will also affect other resources. For example in SQL Server a process called lacy writer has to allocate enough internal memory pages, which requires a lot of processing power and additional disk Input/Output (I/O).[7] According to [16] the best practice in setting the maximum server memory is to set it proportional to the expected capacity or database size but never above the maximum physical memory available. The minimum server memory should be one to two Gigabytes (GB) less than the maximum physical memory to ensure that the operating system has enough memory available.

When identifying memory bottlenecks [41] says that it is important to distinguish whether SQL Server is facing internal or external memory pressure. We talk about external memory pressure when the Database Management System (DBMS) can not get sufficient memory from the system. Internal memory pressure on the other hand, is when the by the DBMS allocated memory, gets used by other processes such as extended stored procedures. Here the previously mentioned Activity Monitor can help to find out where the memory bottleneck comes from. Table 1 taken from [18] lists the most important memory performance counters.

When memory analysis returns a bottleneck [7] suggests the following common resolutions.

- Optimize Application Workload
- Allocate More Memory to SQL Server
- Move in-memory Tables Back to Storage

Counter	Description
Page Life Expectancy	number of seconds a page will stay in the buffer pool without references
Buffer Cache hit ratio	percentage of pages found in the buffer cache without having to read from disk
Page reads and writes / sec	the number of physical database page reads that are issued per second
Lazy writes / sec	the number of buffers written per second by the buffer manager's lazy writer
Memory Grants Pending	the total number of processes waiting for a workspace memory grant
Total Server Memory	the amount of memory the SQL Server is currently using
Target Server Memory	the amount of memory the SQL Server would like to use
Available MBytes	the number of free memory which is available in the system

Table 1: Memory Performance Counters

- Increase System Memory
- Compress Data
- Address Fragmentation

Most of this resolutions can applied by simply increasing physical memory on the used server. Others like for example data compression has to be implemented by using special software. Nevertheless when having a memory bottleneck, database performance will significantly decrease due to a mechanism called swapping. Swapping means that if the main memory is full, disk storage gets used to store data. As stated in [39] disk storage has a access time of about 10 milliseconds where else main memory has an access time of about 0.0001 milliseconds. This means that each I/O operation is about 10^5 times slower when swapping is used.

3.2 PROCESSOR

The Central Processing Unit (CPU) builds the center of a computer system. For a system dedicated to SQL Server the CPU is mainly responsible for doing all the necessary calculations, gathering and delivering data, doing the ordering and grouping. So it is always important that CPU usage doesn't exceed the maximum available amount. As it is for memory, SQL Server also implements counters for monitoring CPU usage, which help us identifying and analyzing CPU pressure on the current instance. Table 2 lists the most important counters according to [19]: In case of dealing with performance bottlenecks due to a

Counter	Description
Processor: %User Time	Percentage of time the processor is spending executing user processes
Processor: % Privileged Time	Percentage of processor time spent for executing kernel commands such as I/O requests
System: Processor Queue Length	number of threads waiting for processor time

Table 2: Processor Performance Counters

high processor workload, [7] suggests to eliminate or reduce cost intensive compiles like the ones needed for stored procedures, to stop unneeded services, to not run unnecessary software or to simply use more or a faster processor.

3.3 NETWORK

The network itself is rarely the hardware resource which is limiting SQL Server workload, as long as the Server is running on the same network as the application does, which is true for most of the use cases in modern applications. When facing network problems in most cases these are due to hardware

resources such as routers or switches. Most of these problems can be identified by using the network monitor tool (Figure 2).

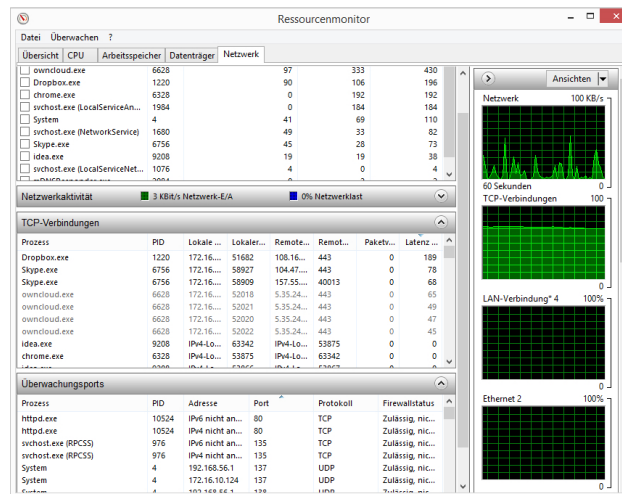


Figure 2: Windows Network Monitor

Nevertheless if network monitor tool does not bring sufficient information SQL Server itself implements the network counters shown in Table 3 taken from [21]: Possible solutions for resolv-

Counter	Description
Bytes Received / sec	receiving byte rate over each network adapter
Bytes Sent / sec	sending byte rate over each network adapter
Current Bandwidth	estimation of current bits per second of the network interface

Table 3: Network Performance Counters

ing memory bottlenecks are:

- Improve Application Architecture
- Update router or switch hardware
- Update router or switch software
- Add more network adapters

3.4 HARD DISK

When talking about hard disks we still talk about one of the slowest parts in a computer system given that the data is stored on magnetic devices. Therefore SQL Server implements also counters for monitoring Disk I/Os. These counters mainly analyze what percentage of time the disk was busy, the rate of read and write operations on disk and the amount of data getting transferred to the disk and back. When facing disk I/O problems the best solutions are:

- install a Redundant Array of Independent Disks (RAID)
- use Solid State Drive (SSD)s as main storage
- use a Storage Area Network (SAN)

A RAID is an union of multiple hard disks in order to improve either performance or reliability. The union of the disks can either be achieved by using a software which is then called a Software RAID. An other way to union the disks is with a special hardware called a RAID controller. This is then called a hardware RAID. According to [40] hardware RAIDs have the better performance as they do not use CPU resources like a software RAID would do. Further RAIDs can be divided into different RAID Levels. According to [39] the following RAID Levels are the most common ones:

- RAID 0 (Striping)
This RAID Level is not having any redundancy in data. It is using n-disks in order to write data in parallel. This means that disk I/Os increase accordingly to the number of disks in the array.
- RAID 1 (Mirroring)
This RAID Level is for having data redundancy. This means that the disk I/Os are not increased and therefore this RAID Level does not improve read/write performance.

- RAID 5

This RAID Level needs at least three disks. With this RAID level redundancy as well as performance gets increased. The data gets distributed on each disk with storing parity on each stripe.

- RAID 10

This RAID Level is a hybrid array. It consist of at least four disks. Here the mechanisms of RAID 1 and RAID 0 are combined to have redundant data with having the performance improve of multiple disks used.

Table 4 shows the improve of disk I/Os when working with different RAID Levels.

RAID Level	Number of Disks	I/O per Disk	total I/Os
RAID 0	2	200	400
RAID 1	2	200	200
RAID 5	3	200	240
RAID 10	4	200	530

Table 4: Disk I/Os by RAID Levels

A SAN is a storage network using the above described RAID technology to even further improve performance by creating multiple RAID groups which are capable of improving reading and writing data to disks. Whenever high I/Os and a large capacity is needed, a SAN is the way to go.

Mainly all this systems use a magnetic media, such as hard drives, to store data. Such medias still have mechanical moving parts limited in performance. Nowadays it is possible to store large amounts of data on different devices such as SSDs.

A SSD disk is a persistent disk drive base on semiconductor technology without having any mechanically moving parts. These kind of data storage distinguish itself by having very high disk I/Os and being very robust. Using this kind of storage

can increase I/O performance significantly. By using this technology in RAIDs or SANs the performance can even be further improved. Figure 3 shows the benchmark difference in read and write operations between a conventional hard disk and a SSD.

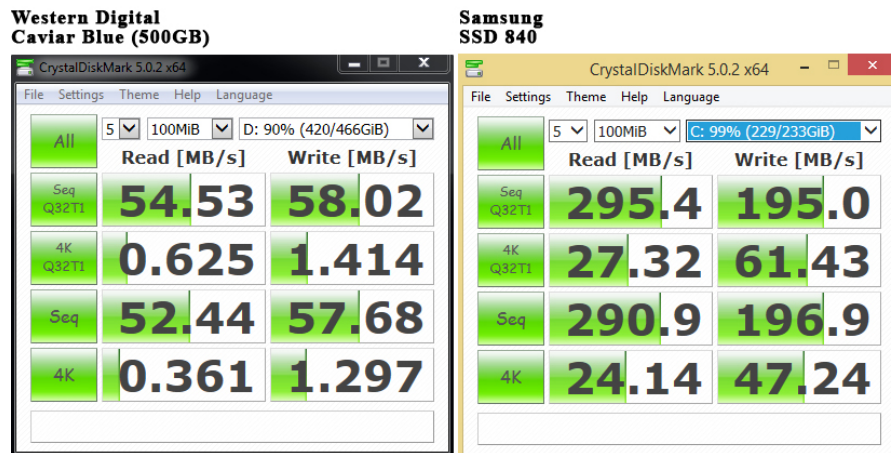


Figure 3: Benchmark Hard Disk Drive vs. SSD Drive

For the benchmark test the CrystalDiskMark software in version 5.0.2 got used. Each test was done with 5 test runs, and a read and write data size of 100 Megabytes. According to CrystalDiskMark company the shown four test results can be interpreted as follows.

- Seq Q32T1: Sequential (Block Size = 128KiB) Read/Write with multiple queues and threads
- 4K Q32T1: Random 4KiB Read/Write with multiple queues and threads
- Seq: Sequential (Block Size = 1MiB) Read/Write with single thread
- 4K: Random 4KiB Read Write with single queue and thread

As we can see the SSD performs about 5 times as fast as a normal hard disk drive meaning that using a SSD as main storage for a database can significantly improve read and write operations and can therefore significantly improve database performance.

DATABASE TYPES

In this section we will have a brief look on which databases can be used to persistently store data. We will further discuss what are the advantages and disadvantages of a certain database type and whether or not they are suitable for the desired purpose. The first database systems got invented in the early 60's, for persisting important data. According to [13] a DBMS must have the following fundamental properties which are essential for persisting data.

- The DBMS must be able to interpret the stored data without the use of special software.
- Only the DBMS is able to access the data and manipulate it. All data access is realized through the DBMS
- The DBMS must be able to organize multi-users and manage the access rights.
- A DBMS must have mechanisms to restore data in case of errors.

As there is a wide variety of different database types, this thesis will focus on the two major types which are important to the in chapter 8 presented use case. Firstly we will discuss relational databases followed by Not only SQL (NoSql) databases.

4.1 RELATIONAL DATABASES

Relational databases use a table model to persist data. As shown in figure 4 each table consists of attributes, also called columns and rows, also called tuples, where the columns make up the data definition and a row represents an entry. Each row needs an unique identifier which is called the primary key. A primary

key is an clustered index and determines how the data is actually stored. We will hear more on indexes in chapter 6. As stated in [2], the backbone of each relational database is build by the relation. A relation is a well defined mathematical concept. The relational algebra is a model which describes how data in a database can be stored, queried and manipulated.

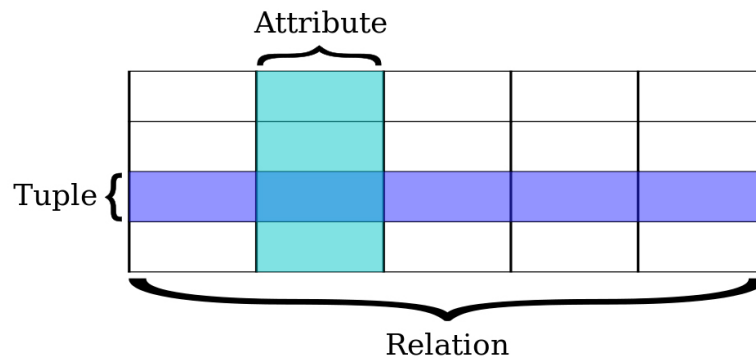


Figure 4: Relational Database Table

A relational database gets used to store data which follow a strict schema. This means that data in a relational database has to be structured and each entry in a database table has to have the same fields with predefined data types. To build a conceptual schema, each object attribute has to be normalized. Normalization is the process to split attributes into multiple relations in order to avoid data redundancy. According to [4] there are four major normal forms.

- First Normalform
- Second Normalform
- Third Normalform
- Boyce-Codd Normalform

In most use cases for building database schemes, the data is called normalized when it is in the third normal form [3]. Further discussion of normal forms would go beyond the scope of this theses and can be read in this article [4].

Using relational databases has a lot of advantages. The first advantage is that the data model in relational databases is very simple. It guarantees data consistency because changes to the data are made on a single point. An other advantage is that due to the fact that each entity gets separated into single pieces or fields, sophisticated queries and operations can be issued against the database. Further there is a standardized language for querying data from each relational database called Structured Query Language ([SQL](#)).

But using a relational database can also have some disadvantages. Due to the inflexible schema of a relational database it is difficult to store constantly changing data. If the fields of an object which should be stored in a database are changing very frequently, the schema has to be adopted on each change. Which in turn is a lot of work and can get very complicated. Further relations like is-a or part-of can not be stored in a relational database because tuples can not consist of sub-tuples. Another major disadvantage is due to the previously mentioned normalization of the data. It can get very difficult in terms of performance to search for complex connections between data or relations.

4.2 NOSQL DATABASES

[NoSql](#) databases combine a wide variety of different database types. Other than relational databases, [NoSql](#) databases were designed to cope with the rapidly rising requirements for data storages in terms of volume, access frequency, performance, agility, scalability and processing needs. As stated in [36], when talking about [NoSql](#) databases, the following four types apply.

- Key-Value stores

Key-Value stores are the simplest [NoSql](#) databases used. Each item stored in a key-value store get represented as an attribute name which is called the key and its value.

- Graph stores
Graph stores are the best option for storing data where each object is related to multiple other objects such as networks or social connections.
- Wide-column stores
As the name already states, these stores are arranged to store columns of data together and are therefore optimized for queries over large data sets.
- Document databases
Document databases are the most common [NoSql](#) databases used. In a document database, each key can have a complex data structure or so called document as its value. Further each document can consist of more documents containing further key-value pairs or even key-array pairs.

[NoSql](#) databases address issues that a relational database is not designed to address. One of the most important issues here is the ability to store large volumes of semi-structured or unstructured data. By not having to use a predefined schema it is easier to make changes to the data structure without having to deal with implementing changes to the schema. Another advantage of [NoSql](#) databases is their ability to easily spread data across multiple database servers without having to implement complex arrangements or having to make hardware to act as a single server as it would be with using a SQL database.

When designing a new system or having to evaluate [NoSql](#) databases the following are the top five considerations to think of [28].

- Data Model
When designing a database one always has to think about the database model. How likely is it that the model changes or how important is the use of a schema? Thoughts about the structure of the data that has to be stored has to be made and afterwards a decision which database type applies best is needed.

- Query Model

The biggest difference in query behavior is that [NoSql](#) databases are more powerful when it comes to efficiently issue sophisticated queries against the data store. Here thoughts on the interconnection between data have to be made and further thinking about the complexity of the needed data queries.

- Consistency Model

Different applications have different requirements for data consistency. Maintaining a consistent data store always has some trade-offs in terms of read and write performance. Therefore considerations regarding the consistency requirements have to be made in advance.

- APIs

Sometimes the calling client for an application can vary. A standardized Application Programming Interface ([API](#)) helps to provide the stored data to any kind of device. When an [API](#) is needed considerations on the ability of the database to support [API](#) development have to be made.

- Commercial Support and Community Strength

When evaluating a database system, the health of the company or the community which backs up the system, should be taken into account. Otherwise one may run into troubles with having no update support and therefore have to use outdated software.

All these points have to be taken into consideration when designing a new application or thinking of reengineering an application which has a poor data retrieval performance.

SQL QUERIES

In this chapter we will have a look on retrieving data from a database server by using [SQL](#) queries. [SQL](#) is a language with which it is possible to edit data as well as the database schema. [SQL](#) is based on the relational algebra which is a formal notation for formulating database schemes as well as data relations. As there are multiple ways to formulate queries it is necessary, in order to keep a system efficient, to know about how queries get processed internally and what can be done to optimize queries. The following section gives an insight in how SQL Server processes queries and shows how queries can be optimized in order to improve data retrieval performance.

5.1 INTERNAL QUERY PROCESSING

Each time a query gets processed, SQL Server is building an execution plan. This plan contains all tables necessary for retrieving the queried data. The creation of such a plan is usually done in four steps. First step is a simple syntax check, to make sure the syntax is correct. If the syntax check passes, SQL Server generates a simple execution tree. The next step is the execution of the so called Algebrizer. The Algebrizer is responsible for analyzing the present data structure as well as checking if table names and data types of the corresponding columns are correct. The output of the Algebrizer process is an optimized syntax tree which gets stored in a special cache called the plan cache. Last step is the Optimizer which generates the final execution plan and handles the order of access to the involved tables [32]. This behavior is also true for ad-hoc queries, queries with parameters and stored procedures.

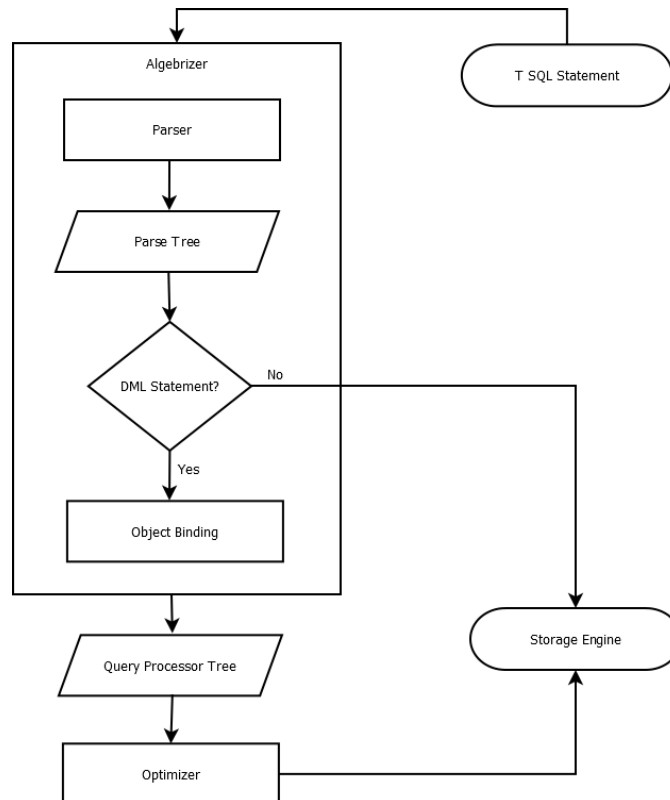


Figure 5: SQL Server Query Execution Optimization [7]

The performance of each query is depended on the generated execution plan. The time a query takes can be summed up by adding the time for generating the execution plan and the time the query itself requires. Taking this into consideration it is very important to keep the cost of creating an execution plan as low as possible. SQL Server logic consists of two main parts, namely:

- The Relational Engine
- The Storage Engine

While the SQL Server Storage Engine is responsible for performing all the data access, data modifications and data caching, the SQL Server Relational Engine does all the parsing, name and type resolutions and optimizations. Figure 5 shows SQL Server techniques to optimize query execution. Whenever a query is submitted the first time, it gets passed to the relational

engine. Within the relational engine the Parser checks the syntax of the query. If a bulk of queries is submitted, the syntax of each query is checked before the processing moves on. If syntax is correct a Parse Tree is generated and passed to the Binding component. There, all the necessary table names, columns, data types and aggregates get resolved and the result is stored in a so called query processor tree. This tree then gets optimized by the Optimizer. The Optimizer first tries to simplify the tree. It checks if all the referenced tables and data is necessary. That being done, SQL Server checks if the execution can be done via a stored trivial plan. If that is true, then no further optimization is necessary. If it is not true, then the query runs over multiple optimization phases and multiple execution plans are generated. If SQL Server has access to more than one [CPU](#), then a phase called parallel plan optimization gets executed. There SQL Server tries to further optimize a query by looking which parts can be executed in parallel. The outcome of this optimization phase is an execution plan which gets stored in the plan cache. There the plan resides for reuse. With this mechanism, the whole optimization process must not be executed again when the same query is issued again [\[7\]](#).

5.2 QUERY PERFORMANCE

In [\[7\]](#) it is stated, that gathering data on query performance is an important but difficult task. Often heavy application workload is responsible for a slow SQL Server performance. Therefore it is important to identify those queries which are stressing the system the most [\[29\]](#). In SQL Server this can be achieved by using Extended Events. Extended Events is a event-handling system for using event data outside the creating process. This is especially useful for all kinds of tracing or logging tools [\[20\]](#). Therefore Extended Events can be used for

- Monitor SQL Queries
- Gather Query Information

- Analyze Query Performance
- Find Deadlocks
- Debug SQL Queries

SQL Server provides a Graphical User Interface (GUI) for using and configuring Extended Events with so called Extended Event Sessions. Figure 6 shows the general window for creating such a logging session. It provides five useful preconfigured templates which can be used for query tuning.

The first is Query Batch Sampling: This collects detailed information of batch and Remote Procedure Call (RPC) statements from 20 percent of the active sessions. If this information is not enough then one could use Query Batch Tracking which collects detailed information of batch and RPC statements from all active sessions. The next template which is useful for analyzing query performance is Query Detail Sampling. This template gathers detailed information of query batches and stored procedures, again of 20 percent of the active sessions. Query Batch Tracking gathers detailed information of query batches and stored procedures of all active sessions. Last template is Query Wait Statistic. This template is designed for collecting internal and external wait statistics of individual query statements, batches and RPCs [34]. When using Extended Events there are a couple of things to keep in mind. First is that it is possible to use debug events. These are designed from Microsoft to debug the SQL Server. As some of these events contain break points it is not wise to use them until you exactly know what you do. Second is that the maximum file size for the information storage is 1 GB by default. This is very small so that if a in depth information gathering is needed, this size will not be sufficient. An acceptable file size here would be around 50 GB. Last point is that the use of the No Event Loss flag should be avoided. This flag prevents SQL Server Extended Events from losing some of the monitored events. As event loss only occurs when SQL Server is under heavy load, the loss of one or another event to keep SQL Server performance up, should be accepted.

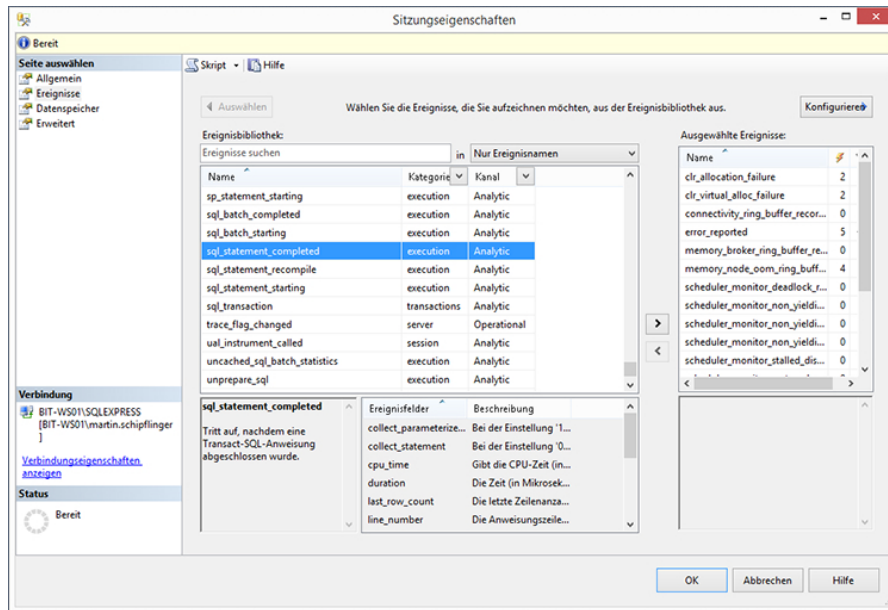


Figure 6: SQL Server Extended Event Session

Once all the detailed information is gathered and stored, this data can be used to analyze query performance. The main goal of this task is to identify costly queries for improving the performance of these queries. This reduces the overall stress on the system and prevents database blocking. In [7] there is a distinction between:

- Costly Queries with Single Execution
- Costly Queries with Multiple Execution and
- Slow Running Queries

Costly Queries with Single Execution often perform a large number of logical reads. To identify those queries the Extended Event Session log can be sorted by the inherent field called *logical_reads*. Costly Queries with Multiple Execution are mainly queries which are not very cost intensive by itself. But as they are executed multiple times they block resources and put pressure on the system. Those queries can not be identified by sorting the log output on *logical_reads*. Here the fields *total_reads* and *total_cpu* are the important ones. By sorting after those fields the queries, which take a lot of reads and CPU time, can

be identified. A big part of application user experience is dependent on whether the system is able to read data effectively or not [8]. The origin of the problem of slow running queries must not always be resource issues. Often other concerns such as blocking or locks come into account. As shown in figure 7

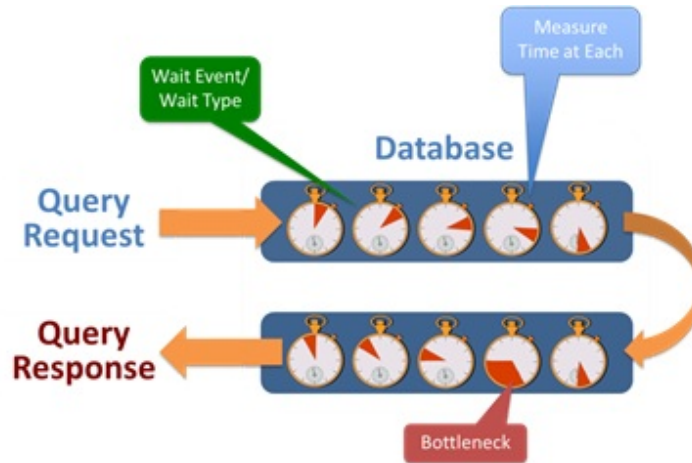


Figure 7: Query Response Time [37]

the response time can be calculated by adding up the processing time and waiting time. So even if processing time is low the response time may be high due to long waiting times. Once all the slow running or costly queries are identified, it is necessary to find out why these queries are performing bad. This can be achieved by looking at the previously mentioned execution plan and analyzing the processing strategy used by SQL Server. Figure 8 shows an execution plan for the following query taken from the e-SMP database.

```
select ITV_Id, i.FDR_Id, f.FDR_Name, ITV_RecallDate from
    Interview as i
inner join Folder as f
on i.FDR_Id = f.FDR_Id
where ITV_RecallDate > DATEADD(YEAR, -2, Getdate())
order by i.ITV_RecallDate
```

Listing 1: SQL Query Example

This execution plan shows the main aspects of query execution. The logical reading direction is from left to right but

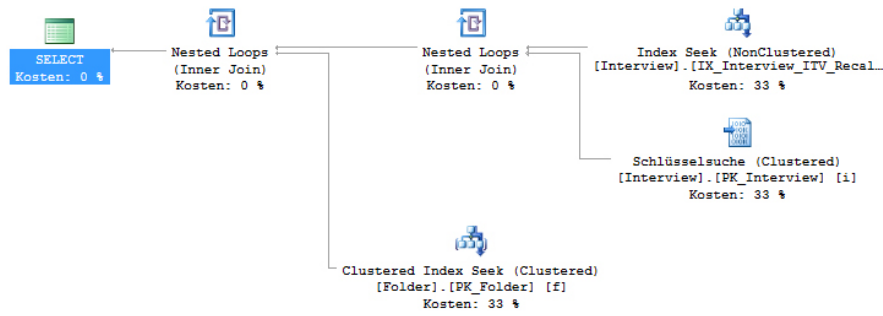


Figure 8: SQL Server Execution Plan

most of the time, when analyzing execution plans, following the physical execution direction, which is from right to left, is the better choice. Here each icon represents an operation and each operation is marked with the percentage of the duration of the whole batch. Further the estimated amount of data transferred is indicated by the thickness of the connector lines. In most cases the execution starts with a data retrieval function which is either a table scan or an index seek. When identifying the costly steps in an execution plan, the following steps can be taken into consideration [7].

- Look at the percentage of total batch cost and start with the most costly ones.
- Analyze the thickness of lines between the operators. These indicate the amount of data transferred.
- Watch for hash joins. If facing large amounts of data, SQL Server often uses hash joins where else when dealing with a small amount, SQL Server uses nested loops.
- Watch for key look-ups. Those can perform a large number of logical reads and can therefore put a lot of stress on the system.
- Analyze sort operations. Sometimes data gets sorted even without an *order by* clause.
- Check if extra operations are performed. Often these can be avoided by rewriting the query.

Taking these steps into consideration it should be possible to figure out where and why a query is performing bad. In the next chapter we will have a look on what can be done to optimize queries and we will introduce some best practices for writing queries.

5.3 OPTIMIZATION

This section will first name and explain some of the most common SQL Server Performance killers. Next it will show some best practices on query design. Finally it will introduce some best practice optimizations for [SQL](#) queries. According to [\[7\]](#) one of the most frequent performance killers for SQL Server is due to poor indexing. If indexing is not done right, SQL Server has to retrieve and process a lot more data and this affects performance of [CPU](#), memory and hard disk. We will have an in depth look at the different kinds of indexes as well as how to correctly use indexes in [Chapter 6](#). Closely connected to improper indexing is a bad query design.

5.3.1 *SQL Query Best Practices*

Query design or the way how queries are written, heavily affects the ability of SQL Server to use the existing indexes. A good query design follows some best practices [\[41\]](#):

- Take advantage of the *NOCOUNT ON* flag
 - This should always be the first statement in batch queries as this avoids that SQL Server returns the number of rows affected after each query execution.
- Define owner of an object
 - Defining the owner of an object avoids that SQL Server has to verify the owner of the object. This reduces the runtime and takes away some stress of the system.
- Search Conditions

- Always be careful with search conditions in queries. Some search conditions can prevent SQL Server from using the index on a table. So the query cost may be very high because instead of an index seek, SQL Server has to perform an index scan which is more ineffective and causes more stress on the system.
- Avoid arithmetic operations or functions in WHERE clauses
 - When arithmetic operations or functions are used on columns in the WHERE clause, the SQL Server Optimizer is no longer using the index of this column. Thus having an index, SQL Server will perform an index scan instead of an less costly index seek.
- Use the Optimizer
 - The standard processing of the SQL Server Optimizer can be overridden by using Optimizer Hints like Join Hints or Index Hints. As in most cases the Optimizer is smart enough to determine which strategy is the most cost efficient. Only in very few cases it will be beneficial to override the standard procedure.
- Data Type Conversions
 - When working with variables always make sure that the data types of the variables are the same as the referring columns. SQL Server is capable of doing implicit conversions but this can again make the use of indexes impossible.
- Views calling Views
 - In SQL Server the Optimizer always assumes that in a view all columns are needed. So the processing behind the scenes is higher than by using an enhanced query.
- Avoid ad-hoc queries

- For each query, SQL Server generates an execution plan and stores it in the plan cache. This is good but absolutely useless if that plan not gets used multiple times. Therefore similar queries should be executed as query with parameters.
- Use native compiled stored procedures
 - SQL Server has the ability to natively compile stored procedures. This transforms the stored procedure form code which has to be interpreted, to a Dynamic Link Library (DLL), which then gives a significant performance improve. This feature is available form SQL Server 2014.
- Transaction tuning
 - Transactions should be kept as short as possible and only containing queries which have to be issued together. It is important that no transaction is able to be left open because this can lead to a waste of resources.

Following these practices in query design helps to decrease stress on CPU, memory, hard disk and network and as a consequence increases the performance of a SQL Server instance. But beside this practices there are some other improvements which can be implemented to improve performance.

5.3.2 SQL Query Optimization

The first easy to implement optimization would be to use full qualified identifiers for table names. Listing 2 shows a correct but not ideal query. This query would only work if the standard schema is used. Where else the query in listing 3 is a query with full qualified identifiers. In this case DBO is the default schema so both of the queries would work but if one of the tables would be in a different schema then listing 2 would throw an error.

```
select * from Note as n
inner join NoteStatuts as ns
on n.NST_ID = ns.NST_ID
where NOT_Type = 'OVER'
order by NOT_Date
```

Listing 2: SQL Query without Identifiers

```
select * from dbo.Note as n
inner join dbo.NoteStatuts as ns
on n.NST_ID = ns.NST_ID
where n.NOT_Type = 'OVER'
order by n.NOT_Date
```

Listing 3: SQL Query with Identifiers

Once all queries are written with full qualified identifiers it would be advisable to reduce the amount of data in the result. As stated in Chapter 5.1, after each query execution the result set is hold in-memory for further processing. Taking this into consideration, reducing the amount of data as soon as possible would be beneficial for memory consumption as well as for processing time. Therefore data should be filtered horizontally, meaning that unnecessary rows get eliminated as well as vertically, meaning that only necessary columns get selected [32]. Vertical filtering can be achieved by putting only those column names in the select statement which are absolutely necessary. Horizontal filtering can be a little trickier. For example Listing 4 and Listing 5 would exactly return the same result set. But there is a significant difference between those two. In Listing 5 the result set gets reduced before the grouping is done. SQL Server has less rows to process and therefore the stress on the system is low. Where else in Listing 4 the grouping is done on the whole result set and rows get reduced not until the *Having* clause. Which in turn puts more pressure on the system.

```
select NOT_Label, count(*) from dbo.Note as n
```



```
inner join dbo.NoteStatuts as ns
on n.NST_ID = ns.NST_ID
group by n.NOT_Label
having n.NOT_Label like 'Ton%'
```

Listing 4: Late Row Reduction

```
select NOT_Label, count(*) from dbo.Note as n
inner join dbo.NoteStatuts as ns
on n.NST_ID = ns.NST_ID
where n.NOT_Label like 'Ton%'
group by n.NOT_Label
```

Listing 5: Early Row Reduction

In the next chapter we will have an in depth look on indexes but for query optimization it is important that queries are written in a way that SQL Server uses the existing indexes. Often invalid Search Argument ([SARG](#))s are used. A [SARG](#) is valid when the following conditions are met [32]:

- All index columns are isolated on one side of the relational operator.
- There is no use of negated relational operators such as *NOT IN*, *NOT EXISTS*, *NOT LIKE*.
- If there is a *LIKE* operator in the *WHERE* clause, the beginning of the compared character sequence should be constant and not start with a placeholder like %.
- Index columns should not be used as function parameters in *WHERE* clauses.

Violating one of those conditions does not mean that the index is completely useless, but it requires an index scan, which is still more effective than a table scan but also less effective than an index seek. Figure 9 shows two execution plans of queries which are returning the same result set. But the second query is much more effective because the [SARG](#) is a valid one where

else the first has the index column as function parameter and is therefore violating one of the above mentioned rules.

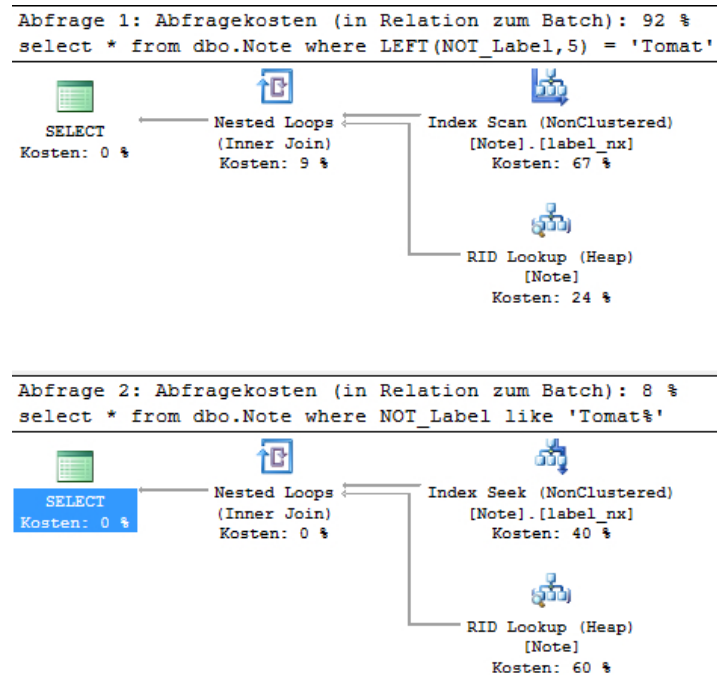


Figure 9: Invalid Search Argument vs Valid Search Argument

When we compare the cost of the batch we can see that if the [SARG](#) is valid and an index seek can be performed the result can be calculated 10 times faster than with an index scan. Having a lot of queries which are performing an index scan or even worst, a table scan, the performance of the system will go down significantly.

The next query optimization is related to sub-queries. Normally the use of sub-queries in [SQL](#) statements should be avoided. But there are some exceptions. The performance of sub-queries is linked to the kind of sub-query that gets used. Single Row sub-queries are good to use because their result is always the same and therefore it has to be executed only once. For all the other executions the result can be read from the cache. Whereas when dealing with correlating sub-queries, meaning that for each row of the result, we get a different sub-query result, this sub-query should be replaced with a join [32].

```

select ITL_id, ITL_Type, 'Light' as Typ
from dbo.Interview_LI
where ITL_Type = '1'
UNION
select ITL_id, ITL_Type, 'Normal' as Typ
from dbo.Interview_LI
where ITL_Type = '2'
UNION
select ITL_id, ITL_Type, 'Heavy' as Typ
from dbo.Interview_LI
where ITL_Type = '3'

```

Listing 6: Union Select

```

select ITL_id, ITL_Type, CASE
WHEN ITL_Type = '1' THEN 'Light'
WHEN ITL_Type = '2' THEN 'Normal'
WHEN ITL_Type = '3' THEN 'Heavy'
END as Typ
from dbo.Interview_LI
where ITL_Type IN ('1','2','3')

```

Listing 7: Case When

Sometimes it is possible to combine multiple queries into one. This often has some advantages like for example that data must not be retrieved multiple times. A well known and often used construct here is the *UNION SELECT*. But using this construct is not as good as it might look. By using a *UNION SELECT* the queries are tied together but they are executed one after each other. This in turn does not bring any processing advantage. Where else using for example a *CASE WHEN* clause, the query has to be executed only once. Listing 6 shows a SQL Statement build with a *UNION SELECT*. This statement will be processed by SQL Server with three table scans. In comparison Listing 7 will return the same result but with only one table scan. Another optimization concerns the storage of temporary data.

When certain results are needed multiple times, it makes sense to store them into temporary tables. When the queried data is needed again only a simple read operation on the temporary table is necessary. This reduces the query overhead whenever this result set is needed to generate another result set. It is also possible to use table variables, which have to be declared beforehand, to store needed result sets for further processing. Once a table variable is declared a *INSERT INTO* or *SELECT INTO* statements could be used to fill the variable with a temporary result set.

The last optimization concerning queries is to transfer database logic onto the database server. By implementing logic on the server the amount of data which has to be transferred between the client and the server can be minimized. One possible way to achieve this is by using views. A view is like a virtual table or a stored query. The view itself is not stored as a table, it is a stored *SELECT* statement [22]. A view itself has no cached execution plan but the optimized syntax tree gets stored. So when the query gets compiled the time for building this optimized syntax tree is saved. Further a lot of previously mentioned optimizations for queries can be implemented and every client who uses this view can take advantage of the optimized query [32]. Another way to transfer logic to the server is the use of functions and stored procedures. For functions and stored procedures SQL Server generates an execution plan and this is stored in the plan cache. If suitable parameters are used for those functions or procedures this plan gets reused very often. This avoids a lot of processing which results in having less stress on the system.

Now that the most effective optimizations in queries are discussed we can move on to the next chapter and have a look on indexes.

INDEXES

In this chapter we will have a look in indexes. In database operations indexes play a very important role. Without indexes all data stored in a database would be an unordered mess. Further without indexes searching for certain parts of data would be ineffective. Often indexes are compared to a library system where each book is an entry in a database. In a library of a certain size it would be very difficult and long lasting to find a book if there would not be some kind of ordering and a table of content. Indexes are the most common way of improving database performance because an index can significantly reduce disk I/Os. Many performance problems can be lead back to missing or bad indexing. Therefore we will first have a look on what kind of indexes are available in SQL Server and where the differences are. Next we will have a look at different indexing strategies as well as some examples of indexes. Finally we will examine how indexes can be analyzed and maintained.

6.1 INDEX TYPES

This section will describe which types of indexes exist. There are three major types of indexes and these three have a different way to store their data. Where Heap Tables and Clustered indexes have an direct impact on how the data gets stored, non-clustered indexes are independent of the underlying row storage [38].

6.1.1 *Heap Tables*

The heap is building the base of indexing. The first time a table gets created the initial storage structure is called the Heap.

Rows in a heap table are numbered by their entry time. Meaning that the first row in the table, is the first row in the index. Until no clustered index gets created on the table the Heap will be the default ordering. That implies that the Heap table index has an direct impact on how data is stored. To sum this up, a Heap table is a database table without a clustered index [7].

6.1.2 *Clustered Index*

Adding a clustered index to a table means that one or more columns get selected and those are now the key columns for this table. Each new entry gets ordered and stored according to the index, meaning that the index directly affects how table data is stored. Taking this fact into consideration this also means that a table can by definition only have one clustered index. Talking about the key selection, there are different kind of keys which can function as index column. The main distinction is between natural keys and surrogate keys. Natural keys are unique keys which exist in the nature, like for example a social security number, whereas surrogate keys are unique made-up keys, like for example an auto increment integer identity. As already mentioned in the previous section, a table must either have a clustered index or it is a Heap table.

6.1.3 *Non-Clustered Index*

Non-clustered indexes are completely different from Heap tables or clustered indexes. Non-clustered indexes are not changing the way the table gets stored on a hard disk drive, they build a new data structure. By building a non-clustered index, the selected column gets ordered and stored in a new data structure holding a reference to the clustered index or Heap location of the original data. In most cases this new data structure is a balanced tree, which is also called B Tree. Figure 10 shows an example of a B+ Tree which is a special version of a

B Tree. B+ Trees are very effective whenever data has to be read from disk. A B+ Tree compared to a binary tree has nodes with many children instead of having exactly two. This fact lowers the depth of the tree and therefore less reads are necessary. A B+ Tree ensures that there will be a fixed maximum number of reads to access any data stored in that tree. The number of reads is depending on the depth of the tree [5].

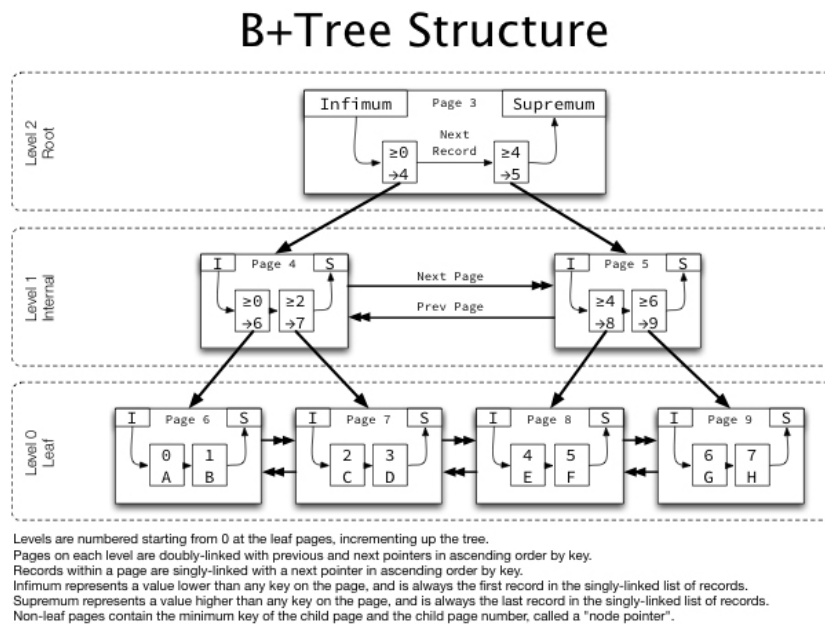


Figure 10: B+ Tree Structure [5]

In a B-Tree data is organized in a hierarchical tree structure. This structure is sorted and optimized for searches. According to [35] the average search cost in a B-Tree is $\mathcal{O}(\log n)$. That means that compute time goes up linearly while n goes up exponentially. In other words if it takes 1 second to search for a result in 1000 entries, it would only take 2 seconds to search for it in 10000 entries.

6.2 INDEX STRATEGIES

When it comes to designing indexes, often only very few indexes get used. When creating indexes it always has to be considered, which index is good for what kind of task [11]. This

section is introducing some index design patterns which should be used for achieving a particular task with an index. It is separated into subsections, each of them showing how and when the referenced index should be used. The first kind of index we will have a look on are Heap table indexes. Next, attributes and patterns for clustered indexes are shown. Finally, some patterns for non-clustered indexes are introduced and described.

6.2.1 *Heap Tables*

By default a table should always have a clustered index, but in some cases using a Heap is also valid. Heaps should mainly be used when dealing with temporary objects. A temporary object would either be a temporary variable or a temporary table. In most cases, when creating a temporary variable or a temporary table, database designers often forget on building an explicit clustered index. Then the table is stored as a Heap table. This is a valid approach but in most cases it is preferable to build a clustered index.

6.2.2 *Clustered Index*

According to [7], a good clustered index should have four characteristics in order to be a well-defined clustered index. These four characteristics are:

- Static
- Narrow
- Unique
- Ever-Increasing

Using a static key column makes sure that the key is not changing when a row gets updated. As stated in the previous chapter, when using a clustered index, rows are stored in the order of the used key. When having a key column which is changing, the

position of the row in the index will also change. This would result in the row, inserted on a different storage page. Further as every non-clustered index is holding a reference to the clustered index key column, every non-clustered index has to be changed accordingly. This can lead to an index fragmentation which in turn is bad for the performance. The narrow attribute says that, the key column for a clustered index should ideally be a single column which has the smallest data type applicable. A reason for this is also that in each non-clustered index the key column of the clustered index is stored. The next attribute which is important for a key column is its uniqueness. A clustered index stores a single row in a single location. SQL Server has a built in unifier which would extend the key by some additional bytes. Using the unifier in turn would lead to a non narrow index. Last point is the ever increasing key. A good index should increase its value with every entry. So new rows are stored at the end of the index and therefore no fragmentation can occur.

Focusing on these premisses, there are a couple of ways to choose a good key column. If the data that has to be stored in a table does not bring a good key, there is always the possibility of using an identity column. This mostly is a numeric value which is increasing itself whenever a new row gets added. Where an identity column is okay for some tables sometimes it is preferable to use a natural key. A natural key is a key which already lives in a data row. So for example a social security number might be a good key column for identifying a person. Natural keys are preferable because they are easier to read and the people which are working with the data can read a referenced natural key more easily than a made up identity. Further a natural key often meets at least 3 of the above mentioned criteria for choosing a good index key. Only the narrowness will sometimes be worse than by using a identity column.

The next pattern is called multiple column. In this pattern two or more columns uniquely identify a row in a table. This is often the case, when two or more tables are related to a third

table as it is in mapping a many-to-many relationship. Here the columns used for the relationship represent the clustering keys.

The last pattern is the Globally Unique Identifier ([GUID](#)) pattern. This pattern is the least effective pattern. As this pattern builds a non integer based uniquely generated value, it solves the problem of generating new keys even when disconnected from the database. SQL Server provides two functions to build a [GUID](#):

- NEWID()
- NEWSEQUENTIALID()

Both of these functions generate a 16-byte hexadecimal value. Compared to an Integer which needs 4-byte this is a very big data type and this violates the narrow characteristic of a good index key. The NEWID() function generates a new [GUID](#) which is unique but they are not increasing. So the chance of running into an index fragmentation is very high. The NEWSEQUENTIALID() function generates [GUID](#) which are increasing but only since the last server restart. So when the server gets restarted, there is no guarantee that the newly generated IDs will be higher than the ones created before. This fact also leads to a fragmented index.

6.2.3 *Non-Clustered Index*

So far we have seen strategies for Heap table indexes and clustered indexes which affect the way data is stored in the database. But we also need a different kind of index, which is not dependent on how the underlying data is stored. As the nature of non-clustered indexes differs from clustered indexes there are a couple of guidelines or considerations which have to be taken into account. These guidelines are different from the ones mentioned in the previous section for clustered indexes. [1] states the four most common considerations when designing non-clustered indexes. The first concerns the frequency of change for the index key columns. Here a rule is that the more

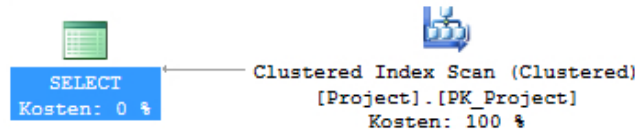
often the data changes, the more often the row in the index has to change its position. That means that the non-clustered index has to be rebuilt frequently which in turn has a negative effect on performance. The second consideration is what is the frequency of the queries this index will improve. The higher the frequency the more effective the index is. The third consideration covers the business needs. Indexes that support critical business operations might be more important than indexes for frequently used queries. Thus the queries for critical business operations should be able to use a proper index. The last consideration deals with the cost of maintaining an index in terms of time and space. It is possible, that the creation and defragmentation of an index is higher than the performance improvement scooped by the use of the index. Following these considerations lead to the following five most important patterns used for Non-Clustered Indexes [38]:

- Search Columns
- Index Intersection
- Multiple Column Indexes
- Covering Indexes
- Filtered Indexes

The first pattern is the Search Columns pattern. This pattern states that a non-clustered index should be set on those columns which are referenced in a search query. If for example in a project database the projects are queried by their name and not by their id then it would be advisable to set an index on the project name column. This reduces the need for a clustered index scan and does instead a non-clustered index seek which is less cost intensive because it minimizes the page reads necessary to retrieve the desired data. Figure 11 taken from the e-SMP database shows the execution plans with and without using a non-clustered index on a search column.

without index on project name

Abfrage 1: Abfragekosten (in Relation zum Batch): 100 %
 select Pro_id, Pro_name from Project where Pro_name = @1;

**with index on project name**

Abfrage 1: Abfragekosten (in Relation zum Batch): 100 %
 select Pro_id, Pro_name from Project where Pro_name = @1;

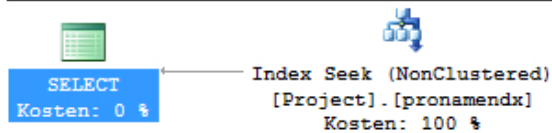


Figure 11: Non-Clustered Index Seek

The next pattern is index intersection. The search column pattern is good if data of only one column is required, but often this is not enough. Sometimes two or more columns are subject to a search query. Here the index intersection pattern has its strengths. When all predicates of the WHERE clause of a query have a non-clustered index, SQL Server is able to find all data based on the index. Hereby SQL Server uses the multiple non-clustered indexes to retrieve the matching rows on the clustering key. This operation is called index intersection [15].

```
create index IX_typedate on Note(NOT_Type, NOT_Date)

SET STATISTICS IO ON

declare @1 as nvarchar(256)
declare @2 as datetime
set @1 = 'SPE'
set @2 = '2015-02-02'
select NOT_Type, NOT_Date from Note
where NOT_Type = @1 and NOT_Date = @2;
SET STATISTICS IO OFF
```

Listing 8: SQL Query with statistics on

The multiple columns pattern is the next pattern which can be used when defining indexes. As the first two patterns focused on a single column as the key column, a non-clustered index can have up to 16 columns as key columns. This is helpful because this can significantly reduce the number of reads necessary to obtain the desired data. If an application for example has a lot of queries containing the same predicates in the WHERE clause it is advisable to put these columns in a single non-clustered index. By analyzing the statistic I/O of the query shown in listing 8, there is a significant difference in needed scans and reads when using a multiple column index. Without the index, two scans and 5 logical reads are necessary to retrieve the desired data, where else by using the index only one scan and two logical reads are required. Further using this pattern can help to reduce the number of indexes needed by wisely putting together the right key columns. Figure 12 shows the differences in execution plans when using multiple columns index.

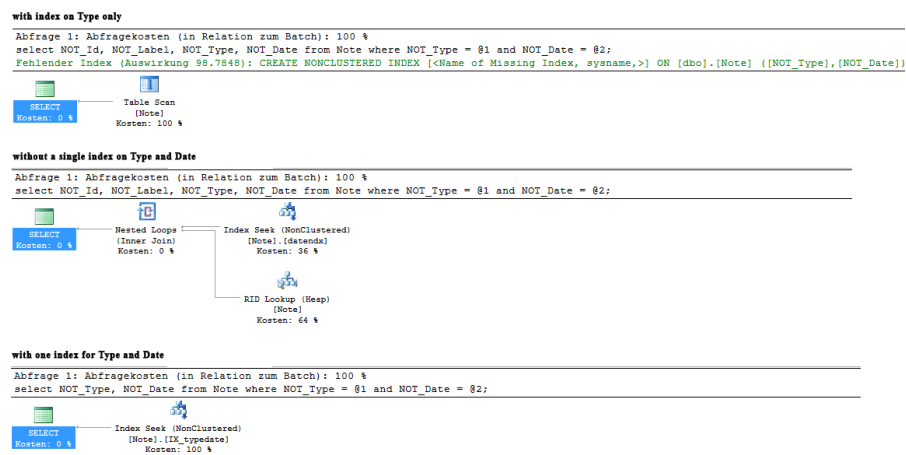


Figure 12: Multiple Columns Index Pattern

When building indexes [7] recommends, that the following points should be taken into consideration:

- JOIN and WHERE clause criteria should be examined
- Indexes should be as NARROW as possible

- The uniqueness of the index column should be examined
- The data type of the index column should be examined
- The type of the index should be considered

6.3 INDEX MAINTENANCE

This section is dedicated to the maintenance of indexes. As indexes are a kind of tables, where data is stored in a different way as it is in the actual table, they have to be maintained. With each data modification the index can change and therefore there is a possibility to not perform at its best. When dealing with index maintenance there are always factors to consider.

- Index fragmentation and
- Index Statistics

The first factor we look at, is index fragmentation. According to [38] an index can get fragmented when one or all of the following operations are frequently used on a table.

- INSERT
- UPDATE
- DELETE
- SHRINKDATABASE

As we can see nearly all write operations can lead to a fragmentation. This means that if the used database is not a read-only database, index fragmentation is an issue, to pay attention to, in order to maintain a performing index. The INSERT operation is the least likely operation that can lead to an index fragmentation. When using an index pattern with a clustered index having an index column with an ever increasing value like an IDENTITY, the index can not get fragmented. Unfortunately

this is not the only pattern used in database design. When having an unique identifier like it is by using the NEWID() function, the ID column is not ever-increasing, meaning that the index can fragment.

The UPDATE operation can lead to an index fragmentation if the updated data is exceeding the page limit. This means that the entry does no longer fit on the current page in the memory. So the page has to be split up into two pages and an index fragmentation comes up.

The DELETE operation can fragment an index by removing entries and therefore leaving gaps in the index. If these gaps do not get reallocated, then pages are no longer physically sequential.

The SHRINKDATABASE operation is as the name says, an operations which aims to shrink the database. For these operations data pages are all the same and they try to release space by rearranging data files which in turn can lead to a fragmented index.

The more fragmented an index becomes the more impact it has on the performance of an index. One of the measure which affects performance the most are I/Os. They are always considered to be a bottleneck in databases and a fragmented index does not reduce I/Os but having a fragmented index due to page splits, a database system needs two I/Os to get the data instead of one. Which in turn means a performance reduce of 50 percent for each index entry which got split. An other effect are contiguous reads. Many DBMS have an operation called read-ahead. This means that the DBMS is able to load pages into memory which are likely to be used, instead of waiting for the I/O operation. If now the index is fragmented in terms of having gaps in the physical order of the entries, data is loaded ahead which is not residing in the index any more. So the DBMS has to change the reading source.

According to [23], when having a fragmented index there are two major solutions to repair the index. The first option is an index reorganization. An index reorganization reorders

the physical order of the pages and closes the gaps which can occur through the previously mentioned operations. This solution is considered the best when having an index fragmentation which makes up less than 30 percent of the index. The second option is a complete index rebuild. Here the DBMS builds a new copy of the index and the old index gets dropped when the rebuild is completed. This is typically done if the fragmentation is considered to be more than 30 percent of the index. For preventing that an index get fragmented to heavily, SQL Server offers two automation mechanisms which can help to maintain an unfragmented index. Maintenance plans help to schedule index reorganizations and rebuilds for removing index fragmentation. An other option are T-SQL Scripts. With those scripts index fragmentation can be effectively removed through predefined statements. Further there are some options for preventing index fragmentation. When creating a new index, a special parameter, called fill factor, can be set. This fill factor determines how much percent gets filled and how much is left empty on every data page for new data. If the data stored in a table is changing frequently then a lower fill factor is recommended. Another option is in setting the data types of fields. When having data types where the length is dependent on the content such as it is for variable character length (`varchar`) field, the length can change over time. This means that if the length of a field is fixed, using a `char` field with a fixed length is better than having a `varchar` field, in terms of preventing fragmentation. Further it is always advisable to set proper default values in fields which can be left blank.

But index maintenance is not only dealing with fragmentation. The second factor we have to deal with is index statistics. Index statistics provide important information for the query optimizer, which is used to generate the execution plans. As already mentioned in chapter 5 execution plans get stored and reused. When index statistics provide wrong information on how the required data can be found, the execution plan is bad and therefore the query is performing bad. For this reason the

index statistics have to be maintained. SQL Server here offers maintenance plans, where tasks for creating and updating index statistics can be planned.

To sum it all up, it is not only important to have the proper index on the proper fields, it is also important to maintain all indexes and avoid index fragmentation and wrong index statistics for keeping high performance indexes.

SUMMARY

In this part we discussed some best practices in maintaining a high-performance data storage system for retrieving data from a database.

The first section was dedicated to hardware. Hardware is building the base of each system. Therefore hardware should be suitable for the desired purpose. Having a hardware bottleneck in one of the mentioned sectors can lead to an unstable and unreliable system. When talking about data retrieval the hard disk or data storage is the bottleneck which should be given the most attention. We saw that combining multiple disks to an array and using flash storage technology in stead of magnetic drives can lead to an significant performance improve for retrieving data.

Sometimes it is useful to think about which data must be stored. Is it either structured or does the data follow no structure rules. Depending on the data, the type of database used, should be considered. Relational databases are around for a long time and are highly developed. But although having having high performance for certain data, [NoSql](#) databases can sometimes be more suitable for the desired purpose. By using special algorithms for indexing such as Lucene, this databases are even able to perform better when it comes to high complexity in queries.

In terms of issuing queries and retrieve data from a database it is important to issue the right queries. There are certainly differences in how queries can be written and how efficient they are. Writing queries in a way the database query optimizer can reuse their execution plans, can significantly increase performance of data retrieval. Further using technologies like Language Integrated Query ([LINQ](#)) and their special ability to

write and prepare SQL statements can facilitate the use of queries and improve the reusability of queries.

The last important point was about using indexes. When not using indexes, the search for a specific entry can be very cost intensive. By using the right index on the right fields, data can be retrieved by doing a cost efficient search on an index instead of having to iterate over the whole table as it is done in a table scan. This reduces the needed disk I/Os and is therefore much more efficient. We also saw that indexes have to be maintained in order to function efficiently. Having a fragmented index can lead to a bad index performance, which can in turn even be slower than scanning the whole table.

Taking those best practices into consideration during the system design and implementing them in the system maintenance routines, will result in a sustainable, maintainable and performing data retrieval system.

Part III

USE CASE STUDY: APPLICATION AND IMPROVEMENT

E-SMP APPLICATION

This chapter is introducing the [e-SMP](#) Application in detail. [e-SMP](#) is an online research infrastructure to conduct and support international nutrition studies. It contains different applications and modules to manage various related sets of data. In this chapter I will firstly mark out the context of the application. Next we will have a look at the architecture and different kinds of modules which are part of the [e-SMP](#). We will see how the application is structured and which components reside inside the application. Afterwards we will focus on the database operations. We will see which technologies are used to issue and process queries. Finalized by an overview of the current performance issues.

8.1 CONTEXT AND OBJECTIVES

In the course of an cooperation with the [IARC](#) I got access to the [e-SMP](#) application in order to examine this application in detail. This data oriented system got designed as data storage for dietary data, collected all over the world by the [IARC](#). The whole system consists of two major parts. The first part is the [GloboDiet Desktop](#) application where dieticians from the [IARC](#) do interviews and collect nutrition data in terms of a [Global Dietary Surveillance](#). In this desktop application the gathered data gets collected and can then be exported to an [Extensible Markup Language \(XML\)](#) file, which then can be imported in the second major part - the [e-SMP](#) server application. In this server infrastructure the data gets summarized and cleaned up. Further the data can get enriched with additional information such as notes or remarks. The enriched data is then prepared for further anal-

ysis and can then be exported into different formats. Figure 13 shows the admin interface of the e-SMP application.



Figure 13: e-SMP Application Front-End View

The current state of the application is facing some serious performance problems. Therefore this thesis is intended to analyze the server component of the application and figure out where these problems come from and give advises for possible improvements. As such system bear a high complexity it has to be divided into smaller parts for not failing due to complexity reasons. As closer described in the next section, the e-SMP application is naturally divided into three subsections. To not go beyond the scope of this thesis and reduce the complexity, this thesis will focus on the Interview Manager (IM) part of the application because this parts seems to have the most crucial impact on performance.

8.2 ARCHITECTURE

As shown in figure 14 the e-SMP consists of three major modules. The first module is the IM. The IM Application provides a platform to manage collected interview data and simplifies standardized operations on those such as import, export, merge or

note management. The second module is the Maintenance and Settings (MS) module. This module is responsible for consolidating all settings available in the system, as well as having all maintenance functions which are required to maintain the system. The third module is the Security and Communication (SC) module. This module holds all important functions, concerning security and communication. As according to the IARC the main issues arise in the IM module, this chapter focuses on this module.

In the IM application data is stored within a specific and secure environment offering the service to separate data from different countries even if they are within the same project. Further it implies different user profiles to access data and enable actions according to the user profile settings.

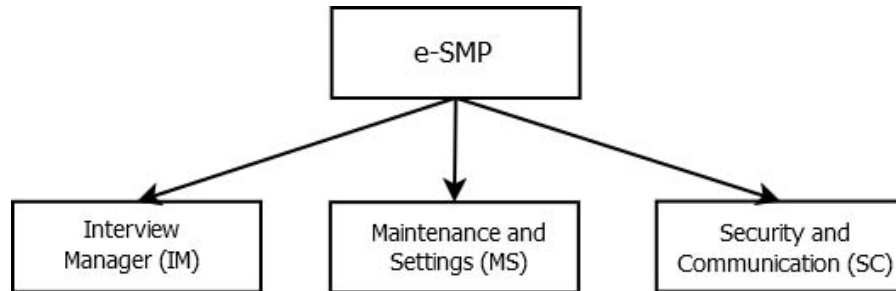


Figure 14: e-SMP Highlevel Architecture

According to [12] this brings two major advantages. The first advantage is the centralization of data. All interviews done on different personal computers by different interviewers get consolidated on one server. This facilitates the control of data integrity as well as offers a synchronized and standardized management interface for all kind of interview processing. The second advantage can be found in the enhancement of communication. The ability to add Notes to every interview simplifies the communication between those who are responsible for the collection of data and those who actually work with and analyze the data. So the interviews and attached comments lead to an improvement process for data quality.

Looking at the code architecture shown in figure 15 we will find 18 .NET platform projects building the whole e-SMP ap-

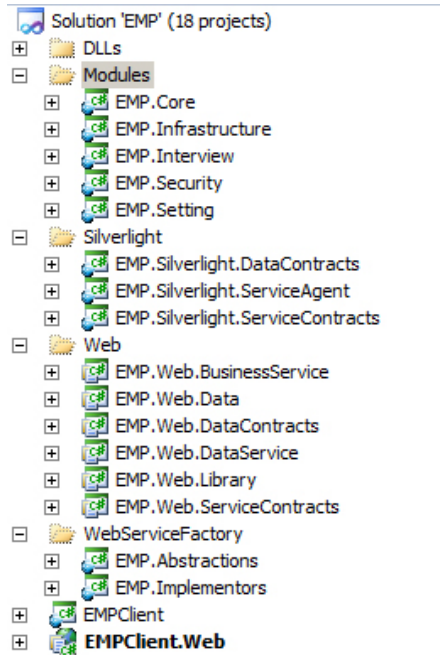


Figure 15: e-SMP Code Architecture

plication based on a Model-View-Controller (MVC) pattern. As stated in [6] the MVC pattern is one of the most widely spread patterns for web presentations. The basic concept is separated into three roles. The first role is the model. The model is basically an object representing information about the domain. It inheres all attributes and is responsible for persisting data. The second role is the view. The view is responsible for representing the information sent by the controller. And the third role is the controller. It takes user inputs, gathers data from the model, manipulates the data if necessary and sends back the data to the view. The main advantage of this pattern resides in the separation of the view and the model. This allows for building applications which are independent when it comes to representing the data. It does not matter if the data gets consumed in a GUI or an API or a Command Line Interface (CLI) client. Model and Controller do not have to be changed in order to represent the results.

In the e-SMP application the view is realized with Silverlight. Silverlight is a plug-in technology developed by Microsoft. It is

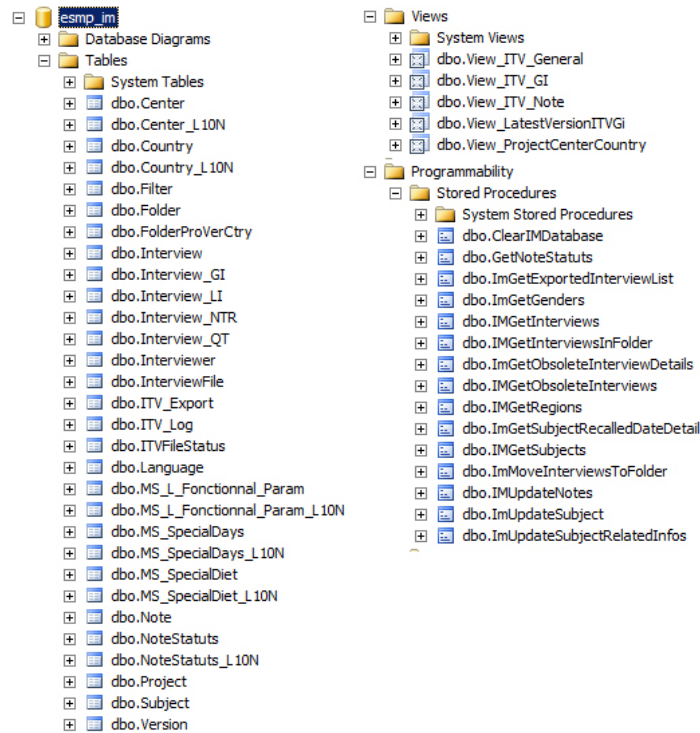


Figure 16: e-SMP Database Architecture

build on the Netscape-Plug-in API ([NPAPI](#)) and it is a development tool for creating web and mobile applications. According to [17] the most important features for Silverlight are:

- IIS Smooth Streaming
 - Enables the delivery of high definition streams with a smooth playback on any devices running Silverlight.
- Deep Zoom
 - Fast and smooth zooming technology for media content.
- Media Format Extensibility
 - Support for a wide variety of third-party codecs.
- Skinning and Styling
 - Easy creation of graphics and controls based on Extensible Application Markup Language ([XAML](#)) templates.
- Pivot Viewer

- Provides easy interaction with massive amounts of data.
- Pixel Shader
 - Provides blur and drop shadows which can be applied to any graphical content.
- Perspective 3D Graphics
 - Content can be applied to a 3D plane where it can be rotated and scaled by the user.

The last component which can be found in the [e-SMP](#) application is the database. The persistent layer is represented by a Microsoft SQL Server 2008 instance. Figure 16 shows the tables, views and stored procedures used, to persist all relevant data generated by or imported into the application.

8.3 QUERIES

In this section, we will have a look on the query structure and query insights of the [e-SMP](#) application. The application uses Microsoft's [LINQ](#) to handle all database communications. [LINQ](#) is a set of features for C# and Visual Basic which allows to ask sophisticated queries against a data set. As shown in Figure 17, [LINQ](#) providers build a layer between the data and the programming language. With this architecture it is possible to abstract the data source and therefore ask queries against different data sources, such as [XML](#) files, relational databases or other data stores.

[LINQ](#) also offers a wide variety of language constructs such as compiled queries. A compiled query is a method offered by LINQ to Entities to improve performance. It takes a query and returns a FUNC that takes a context object and parameters which can be used in the query. A FUNC is delegate for encapsulating a anonymous method. By using this construct it is possible to use the same query with different parameters and therefore this query only has to be compiled once. Listing 9 is

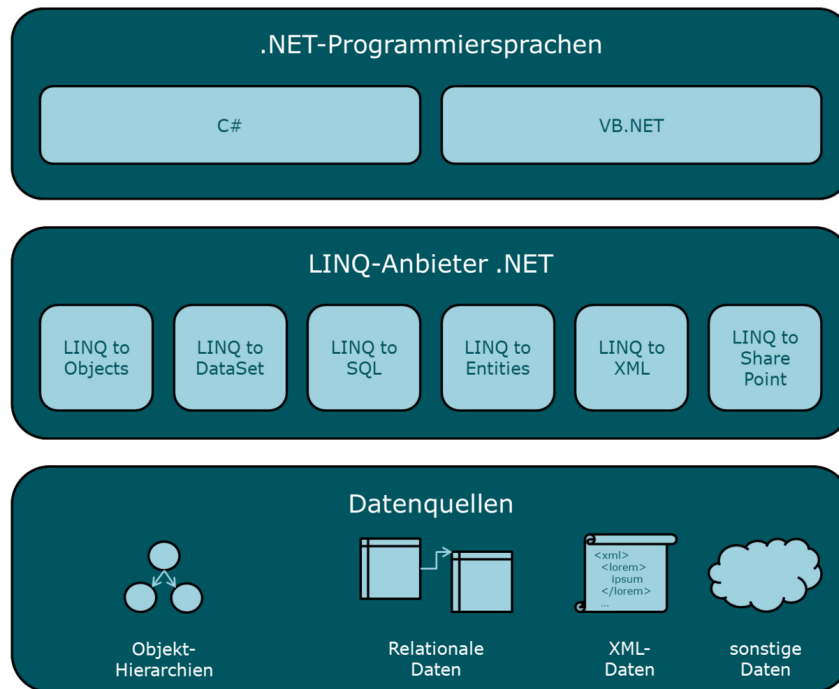


Figure 17: LINQ Architecture [33]

an example of a compiled query for retrieving interviews by version taken from the [e-SMP](#) application.

```

/// <summary>
/// Get interview version will be deleted in the
/// case of restoring
/// </summary>
public static readonly Func<IMEntitiesDataContext,
    int,int, IQueryable<Interview_GI>>
    EMPQueries_GetItvGIList =
    CompiledQuery.Compile<IMEntitiesDataContext, int,int
    , IQueryable<Interview_GI>>((dc, itvId,
    restoreVersion) =>
    from itvGI in dc.Interview_GIs
    where itvGI.ITV_Id == itvId && itvGI.ITG_version
        > restoreVersion
    select itvGI
    );

```

Listing 9: CompiledQuery Example

8.4 ISSUES

The [e-SMP](#) application is facing some serious problems and is, according to the [IARC](#), therefore currently not used. By interviewing the experts of the [IARC](#) and backed up by performing a manual system audit, I was able to tear down the system and find the most crucial system drawbacks. The following list defines, the most important problems which have to be solved in order to maintain an usable system.

- Performance Issues
 - Database Schema
By going through the database schema it turned out that the schema is not well designed. It breaks the data into too many logical pieces, which in turn makes it harder to retrieve the data.
 - Response time in Note screen
As show in figure [18](#) the response time for displaying notes in Note screen takes about a minute. Which makes this section nearly unusable.
 - Pagination
Entries in note view are paginated. But loading the second page also takes about a minute.
 - Filter and Sorting
The Filter and Sorting algorithms in the front end are not well developed. All filtering and sorting takes too long.
- Presentation Issues
 - Response time in export screen
The response time for loading a page in the export screen takes about a minute.
 - Folder Management
A newly created Folder does not get sorted properly.

- Interview sort order
The list of interviews is not sorted on region, subject_code, recalled date.
- Default language
Each project gets a default language, but this language should be proposed to the user first.
- Other Issues
 - Multi user
The IM application is a multi user application. But in order to prevent confusion during updates, a proper locking strategy has to be developed.
 - XML Upload
In case of upload errors, the error gets firstly reported when trying to download the XML file.

✓	Method	File	Domain	Type	Transferred	Size	0 ms	1,36 min	2,73 min	
● 200	POST	EMPDataService.svc	localhost	xml	1,56 kB	1,56 kB				11 ms
● 200	POST	EMPDataService.svc	localhost	xml	0,56 kB	0,56 kB				13 ms
● 200	POST	EMPDataService.svc	localhost	xml	891,66 kB	888,96 kB				58058 ms

Figure 18: Note Retrieval Time

According to the given data, it is quite certain that the issues stated under performance issues have the highest impact on system usability. Thus this thesis will focus on solving some of the issues mentioned under performance issues. As the effort of examining the whole database structure would go beyond of the scope of this work, I will further focus on the issues associated with the retrieval of the queried data.

E-SMP FINDINGS AND IMPROVEMENTS

In this section we will discuss some of the possible improvements which can be implemented in order to enhance performance of the *e-SMP* application and to improve software quality. Firstly we will look at the test and debug environment.

9.1 TEST ENVIRONMENTS

In this theses I used three different environments for testing purposes.

The first one is the actual test system which was provided by the *IARC*. It runs on a virtual box image and uses a 64-bit version of Windows server 2008 R2 as Operating System (*OS*). This machine has four *CPU* cores and a dedicated memory size of four gigabyte. Figure 19 shows a complete list of computing resources used by the virtual machine. The virtual image further has an installation of Microsoft SQL Server 2008 R2 which holds the primary database and it uses the .NET framework 4.5.

The second environment is a desktop system having a quad core *CPU* at a clock speed of 1.73 Gigahertz (*GHz*) and a total main memory of 12 *GB*. It runs a 64-bit version of Windows 10 and has an installation of Microsoft SQL Server 2014 together with an installation of the .NET framework 4.5. It holds the same database as the actual test environment. This environment is primary used for testing the hardware performance of the stored procedures used in the *e-SMP* application.

The third environment is also a desktop system containing a quad core *CPU* at a clock speed of 3.07 *GHz* and a total main memory of 12 *GB*. It runs a 64-bit version of Windows 8.1. This machine has an installation of Microsoft SQL Server 2014 and

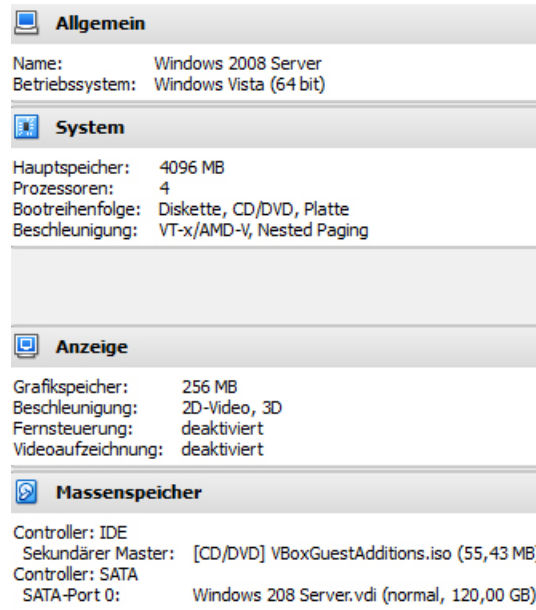


Figure 19: Virtual Box Test Environment

the .NET framework 4.5. It also holds the same database as the actual test environment. This environment is primary used for testing the native compilation feature for stored procedures and the hardware performance in Microsoft SQL Server 2014. All other tests are done in the actual test system and therefore all other measurements relate to this environment.

For testing purposes I used the in Listing 10 shown procedure call. This call contains the default parameter settings used by the *e-SMP* application to retrieve the paginated interview notes.

```
DECLARE @RC int, @UserId int, @UserLogin nvarchar(100),
        @ProjectCode nvarchar(100), @CountryCode nvarchar(100),
        @LanguageCode nvarchar(100), @ProjectId int, @CountryId int,
        @LanguageId int, @IsExportedIncluding bit, @IsITVOrNOT
        bit
        @SortColumnName nvarchar(100), @SortingState nvarchar(100),
        @BirthDateMax datetime, @BirthDateMin datetime, @Classif
        STRING_TABLE
```

```

@ConsumeQty STRING_TABLE, @FacetStr STRING_TABLE, @FolderIds
    INT_TABLE, @GenderCodes STRING_TABLE, @ItvrIds
    INT_TABLE, @Names STRING_TABLE
@NoteType STRING_TABLE, @QtfMethod STRING_TABLE,
    @RecalledDateMax datetime, @RecalledDateMin datetime,
    @RegionIds INT_TABLE, @SBTCodes STRING_TABLE
@SpDayCodes STRING_TABLE, @SpDietCodes STRING_TABLE,
    @StatusIds INT_TABLE, @Types STRING_TABLE, @FromItem int
    , @ToItem int, @output nvarchar(4000)

-- Set parameter values.
SET @output = 'null'
SET @IsExportedIncluding = 'false'
SET @IsITVOrNOT = 'false'
SET @SortingState = 'none'
SET @FromItem = 0
SET @ToItem = 5000
SET @ProjectCode = 'NET-FCS_12'
SET @ProjectId = 129
SET @CountryCode = 'NL_'
SET @CountryId = 5
SET @LanguageCode = 'en-GB'
SET @UserLogin = 'admin'
SET @UserId = 18

EXECUTE @RC = [esmp_im].[dbo].[IMGetInterviews]
    @UserId, @UserLogin ,@ProjectCode ,@CountryCode ,
        @LanguageCode ,@ProjectId ,@CountryId ,@LanguageId
    ,@IsExportedIncluding ,@IsITVOrNOT ,@SortColumnName ,
        @SortingState ,@BirthDateMax ,@BirthDateMin ,
        @Classif
    ,@ConsumeQty ,@FacetStr ,@FolderIds ,@GenderCodes ,
        @ItvrIds ,@Names ,@NoteType ,@QtfMethod ,
        @RecalledDateMax
    ,@RecalledDateMin ,@RegionIds ,@SBTCodes ,@SpDayCodes
        ,@SpDietCodes ,@StatusIds ,@Types ,@FromItem ,
        @ToItem
    ,@output OUTPUT

```


G0

Listing 10: Procedure Call Example

9.2 STORED PROCEDURE IMGETINTERVIEWS

9.2.1 *High Runtime Problem*

The first possible improvement is in improving a stored procedure used in the *e-SMP* application. During the debug process it showed that the application is using a large sized stored procedure to retrieve the interviews from the database. The Silverlight web client is using the *EMPDataProvider* to retrieve the notes from the database and list them in the web client. This *EMPDataProvider* makes a Simple Object Access Protocol (*SOAP*) call to the *IMGetInterviews* function of the *EMPDataProvider* (see figure 21).

Method	File	Domain	Type	Transferred	Size	Time
200 POST	EMPDataService.svc	localhost	xml	1,56 kB	1,56 kB	11 ms
200 POST	EMPDataService.svc	localhost	xml	0,56 kB	0,56 kB	13 ms
200 POST	EMPDataService.svc	localhost	xml	891,66 kB	888,96 kB	58058 ms

Figure 20: Request Time Table for Note Retrieval

This provider executes the stored procedure *IMGetInterviews* which then returns the queried data. As we can see in figure 20, this request takes about 58 seconds.

According to [30], the following three are three major response time borders.

- 0.1 second
 - This is the limit for generating a feeling that the site is reacting instantaneously.
- 1.0 second

- If you want to keep the users attention focused on the current interaction. Here the user will already notice a delay but no special feedback, like for example a waiting bar, is necessary.
- 10 seconds
 - This is the upper border for keeping the users attention on the current dialogue. Above this boarder a user wants to perform other tasks in between. Here feedback on the estimated waiting time is required.

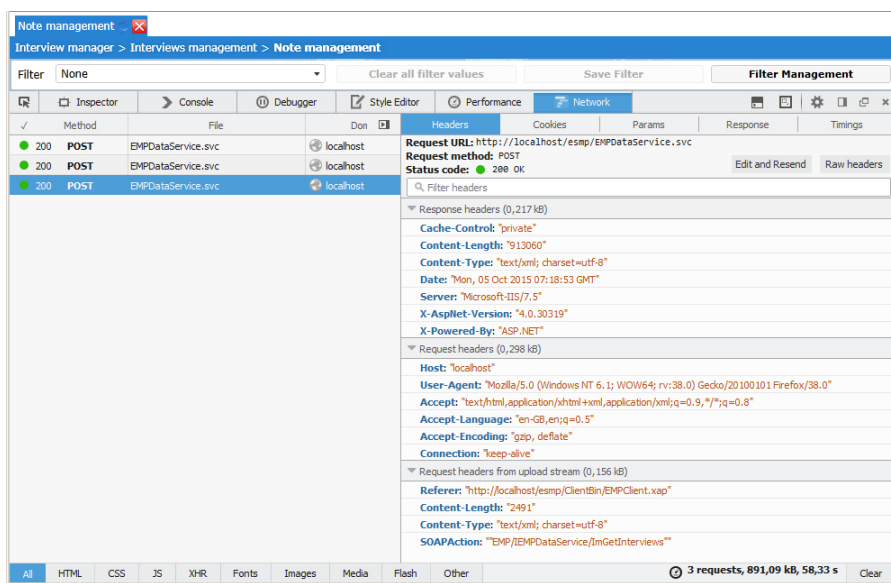


Figure 21: Retrieve Notes

Taking this points into consideration and having a waiting time of about 60 seconds, means that at least a better waiting time indication is required. But for implementing a sustainable improvement this data retrieval has to be further improved.

9.2.2 Possible Improvements

One possible solution would be upgrading the actual Microsoft SQL Server 2008 R2 instance to a Microsoft SQL Server 2014 instance. On the new SQL Server it is possible to natively compile stored procedures. These procedures are Transact-SQL ([T-SQL](#))

stored procedures which get compiled to native code and are then able to access memory-optimized tables. This construct allows the efficient and optimized execution of queries and business logic in the stored procedure which would result in a significant improve of performance [26]. As this feature is relatively new, not all SQL constructs are supported. The following list is an extract of the supported constructs for natively compiled stored procedures as listed in [24].

- Programmability
 - INSERT, UPDATE, DELETE
 - IF and WHILE
 - SELECT
 - RETURN
 - TRY, CATCH, THROW
- Operators
 - Unary operators as +, -
 - Binary operators as *, /, +, -, %
 - Logical operators as AND, OR, NOT
- Functions
 - Math functions
 - Date functions
 - String functions

A test compilation of the IMGetInterviews stored procedure of the e-SMP application was not possible due to the following reason. The procedure as it is now contains the following constructs which are not supported.

- Outer Joins
- Inline Table Variables
- Subqueries

- * Syntax
- Distinct operator
- UNION
- Operators IN, LIKE

To successfully compile this procedure in SQL Server 2014, it has to be rewritten in order to avoid unsupported constructs as listed above. As this is a large procedure with about 800 lines of code, the rewriting would go beyond the scope of this thesis.

Another possibility would be the use of SQL Server 2016 which supports a lot more features as the 2014 version. A full list of supported features can be found in [25]. A comparison of the supported features in SQL Server 2016 and the used constructs in the current stored procedure `IMGetInterviews` shows that the added features close the gap and the native compilation should work in SQL Server 2016.

As mentioned in chapter 3 hardware can play an important role when it comes to processing big amounts of data. As mentioned above, the stored procedure for retrieving the interview notes has a poor performance in terms of runtime. Therefore I tested this procedure on the in Chapter 9.1 mentioned environments with the above mentioned procedure call. Executing this procedure call produced the in table 5 listed results.

Having a deeper look into the SQL Server execution of the stored procedure we will find out, by investigating on the execution plan of the stored procedure `IMGetInterviews`, that this procedure consists of 29 queries. By analyzing the activity monitor mentioned in chapter 3, we will further see the result shown in table 6. It shows that on all test systems, the part where the actual notes get inserted into the virtual table `itv_Note` consumes the most resources in terms of CPU, logical reads and average duration in milliseconds.

Listing 11 shows the extract of the `IMGetInterviews` procedure which is responsible for inserting the data in the virtual `itvNote` table. This query uses multiple sub-queries to insert

Testsystem	runs	avg. runtime in s.
System 1	10	48.11 s
System 2	10	15.56 s
System 3	10	12.70 s
System 1	20	47.16 s
System 2	20	15.23 s
System 3	20	11.70 s
System 1	50	47.15 s
System 2	50	15.22 s
System 3	50	11.70 s

Table 5: Interview Notes Retrieval Time

the desired data. Depending on the used optimizer, replacing the sub-queries with a JOIN may result in a better performance. Further looking at the execution plan of this query shown in figure 22, we will see that this query uses four table scans to retrieve the data. As already heard in chapter 6 table scans are the slowest way to retrieve data from a table. Adding indexes on the right fields of the queried tables results in an improve of query performance.

```

INSERT INTO itv_Note([ITG_Id], [ITL_Id], [NOT_Id],
    [NOT_Type], [NOT_Memo], [NOT_Date], [
    NOT_QuantityConsumed],
    [NST_Id], [NST_Code], [NST_Value])
SELECT DISTINCT [ITG_Id], [ITL_Id], [NOT_Id],(
    select top 1 Name from @notetype_table where
    Code = NOT_Type),
    [NOT_Memo], [NOT_Date],
CASE [NOT_QuantityConsumed]
    WHEN 0 THEN null
    WHEN null THEN null
    ELSE CAST(round([NOT_QuantityConsumed], 1,0)
        as float)
END,

```

Query	CPU (ms/sec)	Logical Reads/s	Average Duration (ms)
System 1			
INSERT INTO itv_Note	371	22414	43356
INSERT INTO interviews	48	935	982
INSERT INTO itv_Li	0	0	988
INSERT INTO itv_Gi	0	0	188
INSERT INTO itv_general	0	0	185
System 2			
INSERT INTO itv_Note	371	22414	43356
INSERT INTO interviews	48	935	982
INSERT INTO itv_Li	0	0	988
INSERT INTO itv_Gi	0	0	188
INSERT INTO itv_general	0	0	185
System 3			
INSERT INTO itv_Note	477	140852	8642
INSERT INTO interviews	14	1728	379
INSERT INTO itv_Li	0	0	175
INSERT INTO itv_Gi	0	0	61
INSERT INTO itv_general	0	0	37

Table 6: Activity Monitor Outcome of IMGetInterviews

```

[NST_Id], [NST_Code], (Select top 1 nt.Status from
    @noteStatus_table nt where nt.Code = n.[NST_Code
    ])
FROM dbo.View_ITV_Note n
WHERE (([ITL_Id] is NULL AND [ITG_Id] IN (select
    ITG_Id from @itv_Gi)) OR ([ITG_Id] is NULL AND
    [ITL_Id] IN (select ITL_Id from @itv_Li)))

```

Listing 11: Insert Into itv_Note from IMGetInterviews

This brings me to the improvement of modifying the stored procedure in a way that the used table variables have a clustered index which gets used by the optimizer and therefore removing some of the above mentioned table scans. Listing

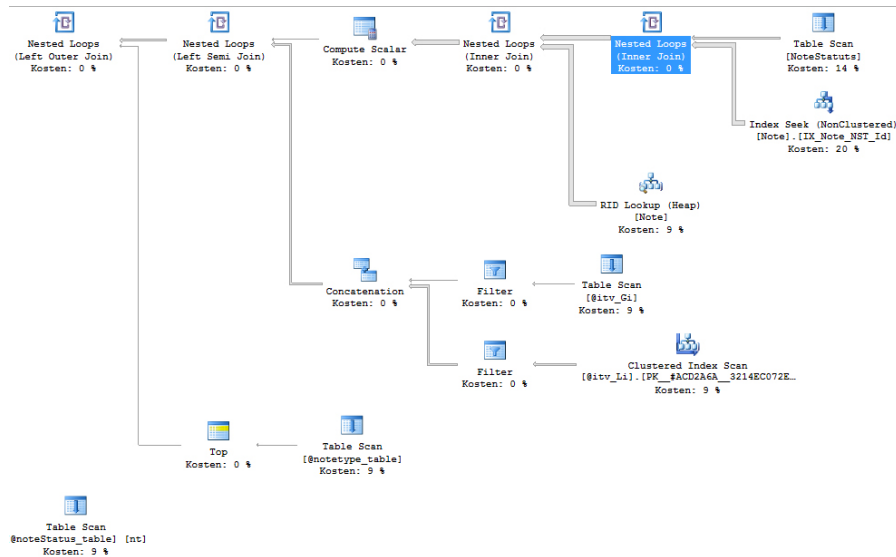


Figure 22: IMGetInterviews Insert Into itv_note

12 shows the modifications done to the two table variables of the IMGetInterviews procedure. By adding a clustered index in terms of adding a primary key to those tables, I was able to replace two table scans with index seeks, which have, as already mentioned in chapter 6, a better performance. Figure 23 shows an extract of the updated execution plan already using the index seek on those tables.

```

DECLARE @notetype_table TABLE (
    Code nvarchar(256) primary key,
    Name nvarchar(256) )

DECLARE @noteStatus_table TABLE (
    Id int primary key,
    Code nvarchar(256),
    Status nvarchar(256) )

```

Listing 12: IMGetInterviews Table Variable Improvement

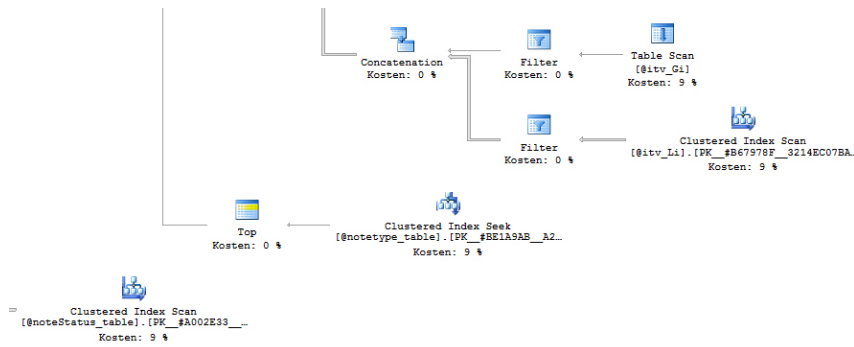


Figure 23: IMGetInterviews Improved Execution Plan

9.3 SQL SERVER VERSION

9.3.1 Outdated SQL Server Version

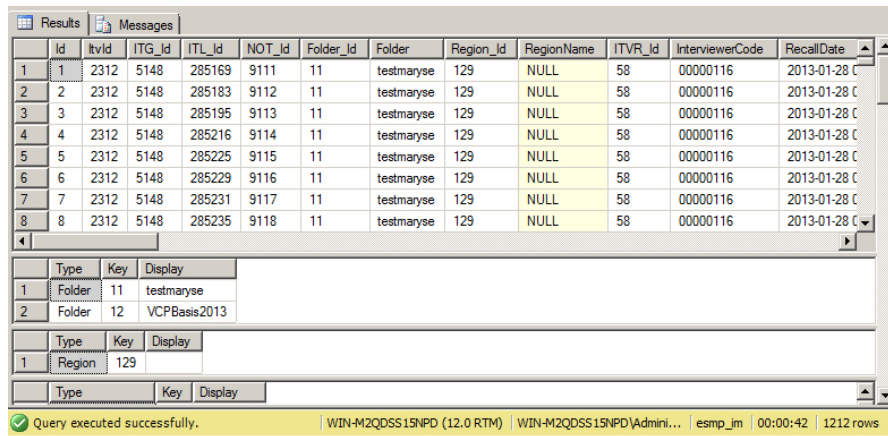
The for the [e-SMP](#) application used database management system is a SQL Server 2008 R2 from Microsoft. This database server is now over 8 years old. As databases evolve very fast and new features and new ways of storing data get integrated into new versions permanently, it may be advisable to think on replacing the old software with a newer one.

9.3.2 Possible Solutions

By trying to replace the old database server, I firstly tried to set up the tech-preview of SQL Server 2016, because this is the newest edition of SQL Server. This resulted in a couple of errors because this version of SQL Server can not be run under the used operating system Windows Server 2008 R2. So I replaced it with the SQL Server 2014. By introducing this replacement, I now have a fourth test system which is using the equivalent hardware of the first system mentioned in [9.1](#) but with an installation of SQL Server 2014.

According to [\[27\]](#) the new installation of SQL Server has the following advantages.

- In-Memory OLTP



The screenshot shows a SQL Server query results window. The main table has 13 columns: Id, ItvId, ITG_Id, ITL_Id, NOT_Id, Folder_Id, Folder, Region_Id, RegionName, ITVR_Id, InterviewerCode, and RecallDate. It contains 8 rows of data. Below the results, there is a schema diagram showing a table with columns Type, Key, and Display. The table has two rows: 'Folder' with key '11' and display 'testmaryse', and 'Folder' with key '12' and display 'VCPBasis2013'. Below the schema diagram, there is another table with columns Type, Key, and Display, containing one row: 'Region' with key '129'. At the bottom, a status bar indicates 'Query executed successfully.' and provides system information: 'WIN-M2QDSS15NPD (12.0 RTM) | WIN-M2QDSS15NPD\Admini... | esmp_im | 00:00:42 | 1212 rows'.

	Id	ItvId	ITG_Id	ITL_Id	NOT_Id	Folder_Id	Folder	Region_Id	RegionName	ITVR_Id	InterviewerCode	RecallDate
1	1	2312	5148	285169	9111	11	testmaryse	129	NULL	58	00000116	2013-01-28 C
2	2	2312	5148	285183	9112	11	testmaryse	129	NULL	58	00000116	2013-01-28 C
3	3	2312	5148	285195	9113	11	testmaryse	129	NULL	58	00000116	2013-01-28 C
4	4	2312	5148	285216	9114	11	testmaryse	129	NULL	58	00000116	2013-01-28 C
5	5	2312	5148	285225	9115	11	testmaryse	129	NULL	58	00000116	2013-01-28 C
6	6	2312	5148	285229	9116	11	testmaryse	129	NULL	58	00000116	2013-01-28 C
7	7	2312	5148	285231	9117	11	testmaryse	129	NULL	58	00000116	2013-01-28 C
8	8	2312	5148	285235	9118	11	testmaryse	129	NULL	58	00000116	2013-01-28 C

Type	Key	Display
Folder	11	testmaryse
Folder	12	VCPBasis2013

Type	Key	Display
Region	129	

Query executed successfully. | WIN-M2QDSS15NPD (12.0 RTM) | WIN-M2QDSS15NPD\Admini... | esmp_im | 00:00:42 | 1212 rows

Figure 24: SQL Server 2014 Improved Timing

- SQL Server is able to keep tables, which get used very frequently, in the main memory. Memory optimized tables are transactional and persistent and can be accessed just like storage based tables, by using T-SQL. Each query can target memory optimized or storage based tables. According to [14] this can make up an performance improve of 5 to 20 percent, depending on some other factors.
- Support for 640 logical processors and 4TB memory
 - With this feature, SQL Server is able to add 640 logical processors and up to 4 Terabytes of main memory to the hardware system.
- Support for 15000 partitions
 - In SQL Server 2008 R2 the number of partitions on tables and indexes is limited to 1000. This means that large tables and indexes can be partitioned in order to be more manageable and scalable. But with having only 1000 partitions available this can lead to a shortcoming on partitions and therefore to performance issues.
- Resource Governor IO governance

- This feature was introduced to get predictable performance in a multi-user environment and to enforce limitations on resources for CPU and memory.
- Buffer Pool Extension to [SSDs](#)
 - With this feature SQL Server is able to handle the integration of persistent random access memory such as solid state drives. As these drives have a very high throughput on disc [I/Os](#). This feature significantly improves [I/O](#) performance.
- Query optimization enhancements
 - Improved optimizer strategies for improving query performance.

After installation and running the the in chapter [9.2](#) described stored procedure it turned out that the new SQL Server improves the stored procedure runtime by about 12 percent. Figure [24](#) shows the returned 1212 rows with an improved runtime of 42 seconds. Issuing the same query multiple times, we see that this result can even be further improved. The results of the test runs can be seen in table [7](#).

Testsystem	runs	avg. runtime in s.
System 4	10	42.06 s
System 4	20	41.52 s
System 4	50	41.49 s

Table 7: Interview Notes Retrieval Time SQL Server 2014

By looking on the technical description of SQL Server 2016 it turns out that this server would theoretically perform even better because of the following points.

- Enhanced in-memory performance
Transaction perform about 30x faster.

- Built-in advanced analytics
Provides more scalability and performance benefits of building and running advanced algorithms.
- Improved Natively Compiled Procedures
Natively Compiled Stored Procedures support more language specific constructs and can therefore be used to compile more sophisticated procedures.

9.4 INTERVIEW PAGINATION

9.4.1 Procedure Call at Each Page

Another improvement can be found by looking on the way how pagination is done in the [e-SMP](#) application. When looking at the in listing 10 shown procedure call we will find two parameters which are responsible for pagination.

- @FromItem
- @ToItem

Varying those parameters and using a pagination of 50 as shown in listing 13, it turns out that the runtime of the procedure does not change whether the result set size is 50 or 5000 entries. This means that not the number of entries makes up the bottleneck, it is the procedure itself.

```
SET @output = 'null'
SET @IsExportedIncluding = 'false'
SET @IsITVOrNOT = 'false'
SET @SortingState = 'none'
SET @FromItem = 0
SET @ToItem = 50
...
```

Listing 13: Varied Request Parameters for IMGetInterviews

By further investigating on the pagination feature of the [IM](#) module the following work-flow came up.

1. Initialize the page
2. Get the first 1000 entries by executing the IMGetInterviews procedure
3. By clicking through the pages, the parameters @FromItem and @ToItem get increased and the next 1000 entries are retrieved.

This means that each time the next page button gets pressed, the IMGetInterviews stored procedure gets executed again with the parameters shown in listing 14. Just the item parameters change. This would be normal behavior except for the long waiting time. Figure 25 shows that on each request there is a waiting time from 60 to 70 seconds.

```
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/
  envelope/">
<s:Body>
<ImGetInterviews xmlns="EMP">
  <request xmlns:d4p1="http://schemas.datacontract.org
    /2004/07/EMP.DataContracts" xmlns:i="http://www.
    w3.org/2001/XMLSchema-instance">
    <d4p1:UserId>18</d4p1:UserId>
    <d4p1:CountryCode>NL</d4p1:CountryCode>
    <d4p1:CountryId>5</d4p1:CountryId>
    <d4p1:LanguageCode>en-GB</d4p1:LanguageCode>
    <d4p1:LanguageId>55</d4p1:LanguageId>
    <d4p1:ProjectCode>NET-FCS_12</
      d4p1:ProjectCode>
    <d4p1:ProjectId>129</d4p1:ProjectId>
    <d4p1:UserLogin>admin</d4p1:UserLogin>
    <d4p1:FromItem>1000</d4p1:FromItem>
    <d4p1:ToItem>1999</d4p1:ToItem>
  </request>
</ImGetInterviews>
```

```
</s:Body>
</s:Envelope>
```

Listing 14: Request Parameters for IMGetInterviews

9.4.2 Possible Solution

A possible improvement here would be to initially load more items as the waiting time stays nearly the same whether you load 1000 or 10000 entries. By holding more entries in the main memory there is no need to call the procedure each time the "next page" button gets pressed. This would significantly improve performance and therefore reduce waiting time for loading pages of interviews.

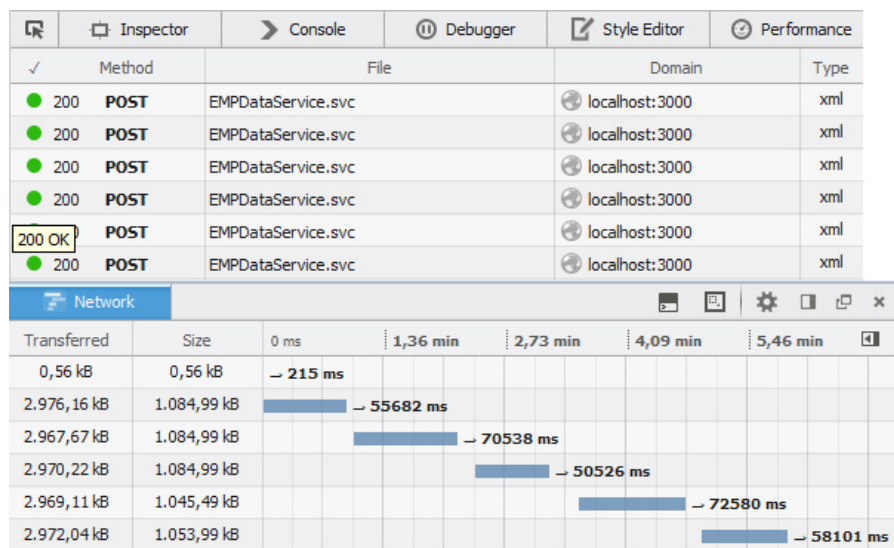


Figure 25: Interview Pagination

9.5 WEBARCHITECTURE

9.5.1 Plug-in technology security issues

The [e-SMP](#) application is build upon a plug-in technology called Silverlight. A reason for not using plug-in technologies in web

application is for security reasons. According to the National Institute of Standards and Technology (NIST) which are maintaining the National Vulnerability Database, Silverlight is facing some serious security issues [31]. Table 8 shows an extract of the Common Vulnerabilities and Exposures (CVE) for Silverlight. Each entry is rated with a Common Vulnerability Scoring System (CVSS) score. The CVSS is a framework for the exchange of characteristics and impacts of Information Technology (IT) vulnerabilities. The scores range from 0.0 to 10.0 and have the in table 9 pictured meanings.

CVE Code	Vulnerability Description	CVSS
CVE-2015-2464	Allows remote attackers to execute arbitrary code via crafted TrueType font (TrueType Font Parsing Vulnerability)	9.3
CVE-2015-1715	Allows remote attackers to bypass intended integrity-level (Silverlight Out of Browser Application Vulnerability)	9.3
CVE-2014-0319	Allows attackers to bypass the DEP and ASLR protection (Silverlight DEP/ASLR Bypass Vulnerability)	7.1
CVE-2013-3896	Improper pointer validation, which allows attackers to obtain sensitive information	4.3
CVE-2013-3178	Improper initialization of arrays, which allows remote attackers to execute arbitrary code or cause Denial of Service (Null Pointer Vulnerability)	9.3

Table 8: Vulnerability Database Entries for the Silverlight plug-in

As we can see four out of five issues have a HIGH severity. This means that these are serious security threats and intruders can use this hints to attack the application and even get sensitive data out of the application. Further Silverlight is build on the NPAPI. According to [10], the NPAPI is a security thread to

web browsers and got therefore banned from Google's Chrome browser. This means that when using Google's Chrome browser you have to set a custom settings flag to allow Chrome to use plug-ins like Silverlight or Unity. This means, that if a user uses Chrome Browser, he will by default not be able to use the [e-SMP](#) application.

Score	Severity Level
0.0 - 3.9	"LOW" severity
4.0 - 6.9	"MEDIUM" severity
7.0 - 10.0	"HIGH" severity

Table 9: CVSS Score Severity Level

9.5.2 Possible Solutions

Here my recommendation would be to avoid plug-in technologies. The for the [e-SMP](#) application used plug-in Silverlight can be replaced by modern, state of the art front-end technologies. Modern front-ends are build upon Hyper Text Markup Language ([HTML](#)), Cascading Style Sheets ([CSS](#)) and JavaScript ([JS](#)). HTML is the primary language for structuring web pages, CSS makes up the front-end style and with JavaScript needed front-end logic can be implemented. Further the use of modern front-end frameworks such as Angular.js should be considered. With Angular.js it is possible to introduce a Model-View-Controller pattern for front-end development. This means that the front-end can be structured according to the MVC pattern and stays therefore more maintainable. Additionally Angular.js introduces a construct called two-way-databinding which lets a user update the model in the front-end. This updates can then be sent to the server via an Asynchronous JavaScript and XML ([AJAX](#)) call. Ajax calls have the advantage of not blocking the front-end until the call is finished. This means that the front-end stays responsive for user inputs although the front-end is communicating with the server and writing data to it. Another point

why such a framework should be used is that it allows to easily build and run sophisticated tasks in the front-end, using the processing power of the front-end machine and therefore reducing server load and improve server performance.

An additional benefit from leaving the plug-in technology behind and switching to modern web technology is that the application can be run on all devices [9]. Whether it is an Android, IOS or Windows mobile device or a Linux, Mac OS or Windows desktop. The application can be run in every Browser on any platform.

SUMMARY

As we saw in the previous chapter, the **IM** module of the **e-SMP** application has a couple of drawbacks which have to be solved in order to get a stable, reliable and responsive application. The biggest issue to the performance of the application is the **IMGetInterviews** stored procedure which is responsible for retrieving all interviews and interview notes. By investigating the application on the in part II mentioned best practices concerning hardware, queries and indexes I was able to produce the in figure 26 summarized outcome.

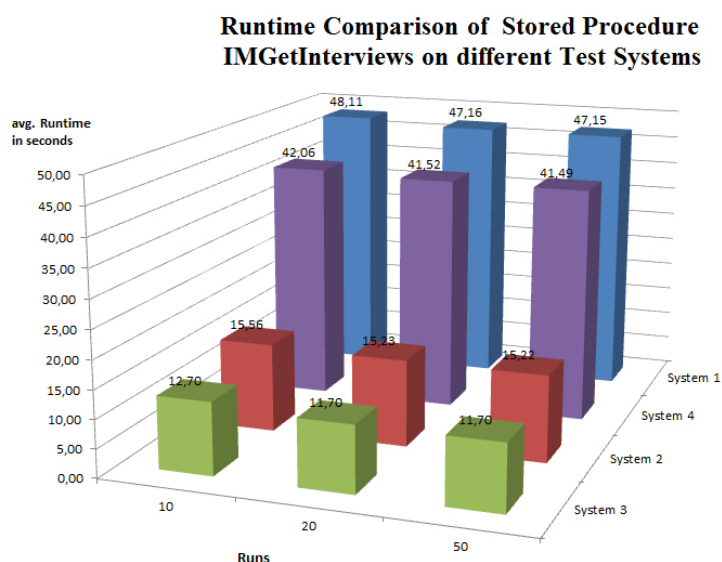


Figure 26: Runtime Comparison of Stored Procedure **IMGetInterviews**

I was able to reduce processing speed of the major stored procedure by 400 percent from a runtime of 47.15 seconds to a runtime of 12.7 seconds in the from the slowest test environment to the fastest test environment. This improve is due to the following improves.

- Improved processing power through adding more cores.
- Improved memory through increasing main memory capacity.
- Improved disk I/Os through the use of an SSD device.
- Improved software version of SQL Server.
- Improved query execution through better indexing.

All these improves can be realized without rebuilding the application. Unfortunately, some of the additional improves mentioned in chapter 9 would require a complete reengineering of the application.

Part IV

CONCLUSIONS AND FUTURE WORK

CONCLUSIONS

By reviewing and debugging the in chapter 8 introduced [e-SMP](#) application, it turned out that the system is not performing good. My first thoughts on the source code being the problem turned out to be wrong. By having a deeper look into the used data retrieval techniques it turned out that the data retrieval mechanisms are poorly implemented. The first big part of improving the system would be to work on the stored procedure `IMGetInterviews`. Here the in chapter [9.2](#) implemented improve to the stored procedure already brought a considerable increase in performance. But this can be further improved by reengineering the whole procedure or implementing a different retrieving strategy.

Another big part which has to be taken into consideration, is to think about the used data model. Investigations on the underlying model show that it is very complex and in order to retrieve data, highly sophisticated queries are needed. When sticking to a relational database, thoughts on how the model can be simplified should be made. An other option would be to switch to one of the in chapter [4.2](#) mentioned [NoSql](#) databases and take advantage of the improved indexing abilities as well as the ability to use the superior query possibilities of such a database.

When residing with a relational database, significant improves can be achieved when updating to the newest version of SQL Server. The significance of this update is reflected by the in chapter [9.3](#) stated outcome and the resulting performance increase.

Looking at the used technologies in terms of modern web development, it turns out that some of the used technologies are outdated. Replacing the front-end with actual technologies

would increase the possibilities in terms of compatibility and performance. By using highly sophisticated and well tested front end frameworks instead of outdated plug-in technology, the error susceptibility would be reduced and the stability of the system gets increased.

All this brings me to the answer of the in chapter 1 stated question: Does the in the e-SMP application used data retrieval techniques follow best practices and are they suitable for the desired purpose. I would say, that the used technologies are in some parts not following best practices and have therefore be improved, in order to get a maintainable, sustainable and performing system. The use of state-of-the-art technology should be considered and improving the underlying hardware should be approached.

FUTURE WORK

In my opinion the next steps to take are to evaluate if it is more reasonable to improve the current system or to make a complete rebuild of the [e-SMP](#) server application with modern, state-of-the-art web- and server technologies. As this application acts as a data store for very sensible and important data, I personally would go for a complete rebuild. As already stated, an improvement of the current system should result in a rebuild of the data model. As this already makes a great deal of work, it is advisable to invest a bit more and therefore get a new, performing and lasting solution which can be sustainable maintained and which is well-equipped for future requirements.

BIBLIOGRAPHY

- [1] Itzik Ben-Gan, Dejan Sarka, Adam Machanic, and Kevin Farlee. *T-SQL Querying*. Microsoft Press, Redmond, USA, 1st edition, 2015.
- [2] E.F Codd. A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, June 1969.
- [3] E.F Codd. Further normalization of the data base relational model. *Courant Computer Science Symposia*, May 1971.
- [4] E.F Codd. Recent investigations in relational data base systems. *IFIP Congress*, June 1974.
- [5] Jeremy Cole. B+ tree Index Structures in Innodb. *Stanford*, August 2013.
- [6] Martin Fowler. *Patterns of Enterprise Application Architecture*. Pearson Inc., Boston, USA, 1st edition, 2003.
- [7] Grant Fritchey. *SQL Server Query Performance Tuning*. Apress, New York, USA, 1st edition, 2014.
- [8] Jesse James Garrett. *The Elements of User Experience*. Pearson Inc., Berkeley, USA, 2nd edition, 2011.
- [9] Peter Gasston. *Moderne Webentwicklung - Techniken und Trends in HTML5, CSS3 und JavaScript*. dpunkt Verlag, Heidelberg, Germany, 1st edition, 2014.
- [10] Google. Plugin-based content doesn't work on chrome, sep 2015. URL <https://support.google.com/chrome/answer/6213033?hl=de>.
- [11] Tapio Lahdenmäki and Michael Leach. *Relational Database Index Design and the Optimizers*. John Wiley Inc, New Jersey, USA, 1st edition, 2005.

- [12] IARC Lyon. e-smp: Interview manager. Presentation, 2015. Presentation on the e-SMP Features.
- [13] Günther Matthiessen and Michael Unterstein. *Relationale Datenbanken und SQL*. Addison-Wesley, Munic, Germany, 1st edition, 2000.
- [14] Microsoft. In-memory oltp (in-memory optimization), sep 2015. URL <https://msdn.microsoft.com/en-us/library/dn133186.aspx>.
- [15] Microsoft. Forceseek index intersection, may 2015. URL [https://technet.microsoft.com/en-us/library/bb510478\(v=sql.105\).aspx](https://technet.microsoft.com/en-us/library/bb510478(v=sql.105).aspx).
- [16] Microsoft. Server memory server configuration options, may 2015. URL <https://msdn.microsoft.com/de-at/library/ms178067.aspx>.
- [17] Microsoft. Top silverlight features, sep 2015. URL <https://www.microsoft.com/silverlight/features/>.
- [18] Microsoft. Sql server, buffer manager object, may 2015. URL <https://technet.microsoft.com/en-us/library/ms189628.aspx>.
- [19] Microsoft. Sql server, buffer manager object, may 2015. URL <https://technet.microsoft.com/en-us/library/ms189628.aspx>.
- [20] Microsoft. Introducing sql server extended events, may 2015. URL [https://technet.microsoft.com/en-us/library/bb630354\(v=sql.105\).aspx](https://technet.microsoft.com/en-us/library/bb630354(v=sql.105).aspx).
- [21] Microsoft. Network interface object, may 2015. URL <https://msdn.microsoft.com/en-us/library/ms803962.aspx>.
- [22] Microsoft. Sql views, may 2015. URL [https://technet.microsoft.com/en-us/library/aa214068\(v=sql.80\).aspx](https://technet.microsoft.com/en-us/library/aa214068(v=sql.80).aspx).

- [23] Microsoft. Reorganize and rebuild indexes, sep 2015. URL <https://msdn.microsoft.com/en-us/library/ms189858.aspx>.
- [24] Microsoft. Supported constructs in natively compiled stored procedures, sep 2015. URL [https://msdn.microsoft.com/en-us/library/dn452279\(v=sql.120\).aspx](https://msdn.microsoft.com/en-us/library/dn452279(v=sql.120).aspx).
- [25] Microsoft. Supported constructs in natively compiled stored procedures, sep 2015. URL <https://msdn.microsoft.com/en-us/library/dn452279.aspx>.
- [26] Microsoft. Introduction to natively compiled stored procedures, sep 2015. URL <https://msdn.microsoft.com/en-us/library/dn133184.aspx>.
- [27] Microsoft. Sql server 2014: Benefits of upgrading from sql server 2005, 2008, 2008r2 and 2014, sep 2015. URL <http://blogs.technet.com/b/cansql/archive/2014/04/14/sql-server-2014-benefits-of-upgrading-from-sql-server-2005-2008-2008r2-and-2012.aspx>.
- [28] MongoDB. Top 5 considerations when evaluating nosql databases. White Paper, 2015. White Paper from MongoDB.
- [29] Benjamin Nevarez. *Microsoft SQL Server 2014 - Query Tuning and Optimization*. Mc Graw Hill, New York, USA, 1st edition, 2014.
- [30] Jacob Nielsen. *Usability Engineering*. Academic Press, Inc., Cambridge, USA, 1st edition, 1993.
- [31] National Institute of Standards and Technology. National vulnerability database, sep 2015. URL https://web.nvd.nist.gov/view/vuln/search-results?query=silverlight&search_type=all&cves=on&startIndex=0.
- [32] Robert Panther. *SQL-Abfragen optimieren*. entwickler.press, Paderborn, Germany, 1st edition, 2014.

- [33] Paolo Pialorsi. *Microsoft SharePoint 2010*. O'Reilly, Köln, Germany, 1st edition, 2011.
- [34] Ray Rankins, Paul Bertucci, Chris Gellelli, and Alex T. Silverstein. *Microsoft SQL Server 2014 Unleashed*. Pearson Inc., New York, USA, 1st edition, 2015.
- [35] Robert Sedgewick. *Algorithmen in Java*. Pearson Inc., Munich, Germany, 3rd edition, 2003.
- [36] Vatika Sharma and Meenu Dave. Sql and nosql databases. *International Journal of Advanced Research in Computer Science and Software Engineering*, August 2012.
- [37] Solarwinds. Response time analysis: How to improve database performance by measuring user experience, may 2015. URL <http://logicalread.solarwinds.com/response-time-analysis/#.VZohqzXtlBc>.
- [38] Jason Strate and Ted Krueger. *Expert Performance Indexing for SQL Server 2012*. Apress, New York, USA, 1st edition, 2012.
- [39] Andrew S. Tannenbaum. *Computerarchitektur*. Pearson, New Jersey, USA, 5th edition, 2006.
- [40] Derek Vadala. *Managing RAID on Linux*. O'Reilly, Cambridge, USA, 1st edition, 2002.
- [41] Steven Wort, Christian Bolton, Michael Cape, and Joshua J Jin. *Professional SQL-Server 2005 Performance Tuning*. Wiley Publishing Inc., Indianapolis, USA, 1st edition, 2008.